*Artificial Intelligence and Language Processing*

*David Waltz Editor*

# A Theory of the Learnable

L. G. VALIANT

***ABSTRACT:*** *Humans appear to be able to learn new concepts without needing to be programmed explicitly in any conventional sense. In this paper we regard learning as the phenomenon of knowledge acquisition in the absence of explicit programming. We give a precise methodology for studying this phenomenon from a computational viewpoint. It consists of choosing an appropriate information gathering mechanism, the learning protocol, and exploring the class of concepts that can be learned using it in a reasonable (polynomial) number of steps. Although inherent algorithmic complexity appears to set serious limits to the range of concepts that can be learned, we show that there are some important nontrivial classes of propositional concepts that can be learned in a realistic sense.*

## 1. INTRODUCTION

Computability theory became possible once precise models became available for modeling the commonplace phenomenon of mechanical calculation. The theory that evolved has been used to explain human experience and to suggest how artificial computing devices should be built. It is also worth studying for its own sake.

The commonplace phenomenon of learning surely merits similar attention. The problem is to discover good models that are interesting to study for their own sake and that promise to be relevant both to explaining human experience and to building devices that can learn. The models should also shed light on the limits of what can be learned, just as computability does on what can be computed.

In this paper we shall say that a program for performing a task has been acquired by *learning* if it has been acquired by any means other than explicit programming. Among human skills some clearly appear to have a genetically preprogrammed element, whereas some others consist of executing an explicit sequence of instructions that has been memorized. There remains a large area of skill acquisition where no such explicit programming is identifiable. It is this area that we describe here as learning. The recognition of familiar objects, such as tables, provides such examples. These skills often have the additional property that, although we have learned them, we find it difficult to articulate what algorithm we are really using. In these cases it would be especially significant if machines could be made to acquire them by learning.

This paper is concerned with precise computational models of the learning phenomenon. We shall restrict ourselves to skills that consist of recognizing whether a *concept* (or predicate) is true or not for given data. We shall say that a concept $Q$ has been learned if a program for recognizing it has been deduced (i.e., by some method other than the acquisition from the outside of the explicit program).

The main contribution of this paper is that it shows that it is possible to design *learning machines* that have all three of the following properties:

1. The machines can provably learn whole classes of concepts. Furthermore, these classes can be characterized.
2. The classes of concepts are appropriate and nontrivial for general-purpose knowledge.
3. The computational process by which the machines deduce the desired programs requires a feasible (i.e., polynomial) number of steps.

A learning machine consists of a *learning protocol* together with a *deduction procedure*. The former specifies the manner in which information is obtained from the outside. The latter is the mechanism by which a correct recognition algorithm for the concept to be learned is deduced. At the broadest level, the suggested *methodology* for studying learning is the following: Define a plausible learning protocol, and investigate the class of con-

This research was supported in part by National Science Foundation grant MCS-83-02385. A preliminary version of this paper appeared in the proceedings of the 16th ACM Symposium on Theory of Computing, Washington, D.C., 1984, 436–445.

© 1984 ACM 0001-0782/84/1100-1134 75¢





Inteligência Artificial e Processamento de Linguagem

Editor David Waltz

# Uma teoria do que pode ser aprendido

L, G. VALENTE

RESUMO: Os humanos parecem ser capazes de aprender novos conceitos sem a necessidade de serem programados explicitamente em qualquer sentido convencional. Neste artigo consideramos a aprendizagem como o fenômeno de aquisição de conhecimento na ausência de programação explícita. Fornece mos uma metodologia precisa para estudar esse fenômeno do ponto de vista computacional. Consiste em escolher um mecanismo apropriado de coleta de informações, o protocolo de aprendizagem, e explorar a classe de conceitos que podem ser aprendidos utilizando-o em um número razoável (polinomial) de etapas. Embora a complexidade algorítmica inerente pareça estabelecer sérios limites à gama de conceitos que podem ser aprendidos, mostramos que existem algumas classes importantes e não triviais de conceitos proposicionais que podem ser aprendidos num sentido realista.

## 1. INTRODUÇÃO

A teoria da computabilidade tornou-se possível quando modelos precisos se tornaram disponíveis para modelar o fenômeno comum do cálculo mecânico. A teoria que evoluiu foi usada para explicar a experiência humana e para sugerir como os dispositivos de computação artificiais deveriam ser construídos. Também vale a pena estudar por si só.

O fenômeno comum da aprendizagem certamente merece atenção semelhante. O problema é descobrir bons modelos que sejam interessantes de estudar por si só e que prometam ser relevantes tanto para explicar a experiência humana como para construir dispositivos que possam aprender. Os modelos também devem esclarecer os li mites do que pode ser aprendido, tal como a computabilidade faz com o que pode ser computado.

Neste artigo diremos que um programa para executar uma tarefa foi adquirido por aprendizagem se tiver sido adquirido por qualquer meio que não seja a programação explícita. Entre as habilidades humanas, algumas parecem claramente ter um elemento geneticamente pré-programado, enquanto alguns outros consistem na execução de uma sequência explícita de instruções que foi memorizada. Resta ainda uma grande área de aquisição de competências onde nenhuma programação explícita é identificável. É esta área que descrevemos aqui como aprendizagem. O reconhecimento de objetos familiares, como tabelas, fornece tais exemplos. Essas habilidades geralmente têm a propriedade adicional de que, embora as tenhamos aprendido, achamos difícil articular qual algoritmo estamos realmente usando. Nestes casos, seria especialmente significativo se as máquinas pudessem ser adquiridas através da aprendizagem.

Este artigo está preocupado com modelos computacionais precisos do fenômeno de aprendizagem. Limitar-nos-emos às competências que consistem em reconhecer se um conceito (ou predicado) é verdadeiro ou não para dados dados. Diremos que um conceito Q foi aprendido se um programa para reconhecê-lo tiver sido deduzido (isto é, por algum método diferente da aquisição do exterior do programa explícito).

A principal contribuição deste artigo é mostrar que é possível projetar máquinas de aprendizagem que possuam todas as três propriedades a seguir:

1. As máquinas podem provavelmente aprender classes inteiras de conceitos. Além disso, essas classes podem ser caracterizadas. 2. As classes de conceitos são apropriadas e não triviais para conhecimento de uso geral. 3. O processo computacional pelo qual as máquinas deduzem os programas desejados requer um número viável (ou seja, polinomial) de etapas.

Uma máquina de aprendizagem consiste em um protocolo de aprendizagem juntamente com um procedimento de dedução. O primeiro especifica a maneira pela qual as informações são obtidas de fora. Este último é o mecanismo pelo qual se deduz um algoritmo de reconhecimento correto do conceito a ser aprendido. No nível mais amplo, a metodologia sugerida para estudar a aprendizagem é a seguinte: definir um protocolo de aprendizagem plausível e investigar a classe de conceitos.

Esta pesquisa foi apoiada em parte pela concessão MCS-83-02385 da National Science Foundation. Uma versão preliminar deste artigo apareceu nos anais do 16º Simpósio ACM sobre Teoria da Computação, Washington, D.C., 1984, 436-445. ©1984ACMO001-0782/84/1100-1134 75¢

cepts for which recognition programs can be deduced in polynomial time using the protocol.

The specific protocols considered in this paper allow for two kinds of information supply. First the learner has access to a supply of typical data that positively exemplify the concept. More precisely, it is assumed that these positive examples have a probabilistic distribution determined arbitrarily by nature. A call of a routine EXAMPLES produces one such positive example. The relative probability of producing different examples is determined by the distribution. The second source of information that may be available is a routine ORACLE. In its most basic version, when presented with data, it will tell the learner whether or not the data positively exemplify the concept.

The major remaining design choice that has to be made is that of knowledge representation. Since our declared aim is to represent general knowledge, it seems almost unavoidable that we use some kind of logic rather than, for example, formal grammars or geometrical constructs. In this paper we shall represent concepts as Boolean functions of a set of propositional variables. The recognition algorithms that we attempt to deduce will be therefore Boolean circuits or expressions.

The adequacy of the propositional calculus for representing knowledge in practical learning systems is clearly a crucial and controversial question. Few would argue that this much power is not necessary. The question is whether it is enough. There are several arguments suggesting that such a system would, at least, be a very good start. First, when one examines the most famous examples of systems that embody preprogrammed knowledge, namely, expert systems such as DENDRAL and MYCIN, essentially no logical notation beyond the propositional calculus is used. Surely it would be overambitious to try to base learning systems on representations that are more powerful than those that have been successfully managed in programmed systems. Second, the results in this paper can be negatively interpreted to suggest that the class of learnable concepts even within the propositional calculus is severely circumscribed. This suggests that the search for extensions to the propositional calculus that have substantially larger learnable classes may be a difficult one.

The positive conclusions of this paper are that there are specific classes of concepts that are learnable in polynomial time using learning protocols of the kind described. These classes can all be characterized by defining the class of programs that recognize them. In each case the programs are special kinds of Boolean expressions. The three classes are (1) conjunctive normal form expressions with a bounded number of literals in each clause, (2) monotone disjunctive normal form expressions, and (3) arbitrary expressions in which each variable occurs just once. In the first of these, no calls of the oracle are necessary. In the last, no access to typical examples is necessary but the oracles need to be more powerful than the one described above.

The deduction procedure will in each case output an expression that with high likelihood closely approximates the expression to be learned. Such an approximate expression never says yes when it should not but may say no on a small fraction of the probability space of positive examples. This fraction can be made arbitrarily small by increasing the runtime of the deduction procedure. Perhaps the main technical discovery contained in the paper is that with this probabilistic notion of learning highly convergent learning is possible for whole classes of Boolean functions. This appears to distinguish this approach from more traditional ones where learning is seen as a process of "inducing" some general rule from information that is insufficient for a reliable deduction to be made. The power of this probabilistic viewpoint is illustrated, for example, by the fact that an arbitrary set of polynomially many positive examples cannot be relied on to determine expressions consisting even of only a single monomial in any reliable way. A more detailed description of this methodological problem can be found in [9].

There is another aspect of our formulation that is worth emphasizing. In a learning system that is about to learn a new concept, there may be an enormous number of propositional variables available. These may be primitive inputs, the values of preprogrammed concepts, or the values of concepts that have been learned previously. We want the complexity of learning the new concept to be related only to the number of variables that may be set in natural examples of it, and not on the cardinality of the universe of available variables. Hence, the questions asked of ORACLE and the values given by EXAMPLES will not be truth assignments to all the variables. In the archetypal case, they will specify an assignment to a subset of the variables that is still sufficient to guarantee the truth of the function.

Whether the classes of learnable Boolean concepts can be extended significantly beyond the three classes given is an interesting question. There is circumstantial evidence from cryptography, however, that the whole class of functions computable by polynomial size circuits is not learnable. Consider a cryptographic scheme that encodes messages by evaluating the function $E_k$ where $k$ specifies the key. Suppose that this scheme is immune to chosen plaintext attack in the sense that, even if the values of $E_k$ are known for polynomially many dynamically chosen inputs, it is computationally infeasible to deduce an algorithm for $E_k$ or for an approximation to it. This is equivalent to saying, however, that $E_k$ is not learnable. The conjectured existence of good cryptographic functions that are easy to compute therefore implies that some easy-to-compute functions are not learnable.

If the class of learnable concepts is as severely limited as suggested by our results, then it would follow that the only way of teaching more complicated concepts is to build them up from such simple ones. Thus, a good teacher would have to identify, name, and sequence these intermediate concepts in the manner of a programmer. The results of *learnability theory* would then indicate the maximum granularity of the single





conceitos para os quais programas de reconhecimento podem ser deduzidos em tempo polinomial usando o protocolo.

Os protocolos específicos considerados neste artigo permitem dois tipos de fornecimento de informações. Primeiro, o aluno tem acesso a um conjunto de dados típicos que exemplificam positivamente o conceito. Mais precisamente, assume-se que estes exemplos positivos têm uma distribuição probabilística determinada arbitrariamente pela natureza. Uma chamada de uma rotina EXEMPLOS produz um exemplo positivo. A probabilidade relativa de produzir exemplos diferentes é determinada pela distribuição. A segunda fonte de informação que pode estar disponível é uma rotina ORACLE. Na sua versão mais básica, quando apresentado com dados, dirá ao aluno se os dados exemplificam positivamente ou não o conceito.

A principal escolha de design que ainda precisa ser feita é a representação do conhecimento. Dado que o nosso objectivo declarado é representar o conhecimento geral, parece quase inevitável que utilizemos algum tipo de lógica em vez de, por exemplo, gramáticas formais ou c onstruções geométricas. Neste artigo representaremos conceitos como funções booleanas de um conjunto de variáveis proposicionais. Os algoritmos de reconheci mento que tentamos deduzir serão, portanto, circuitos ou expressões booleanas.

A adequação do cálculo proposicional para re presentar o conhecimento em sistemas de aprendizagem prática é claramente uma questão crucial e controversa. Poucos argumentariam que tanto poder não é necessário. A questão é se isso é suficiente. Existem vários argu mentos que sugerem que tal sistema seria, pelo menos, um bom começo. Primeiro, quando examinamos os exemplos mais famosos de sistemas que incorporam conhecimento pré-programado, nomeadamente sistemas especialistas como DENDRAL e MYCIN, essencialmente nenhuma notação lógica além do cálculo proposicional é usada. Certamente seria demasiado ambicioso tentar basear os sistemas de aprendizagem em representações que sejam mais poderosas do que aquelas que foram geridas com sucesso em sistemas programados. Em segundo lugar, os resultados deste artigo podem ser interpretados negativamente para sugerir que a classe de conceitos que podem ser aprendidos, mesmo dentro do cálculo proposicional, é severamente circunscrita. Isto sugere que a busca por extensões para o cálculo proposicional que tenham classes aprendíveis substancialmente maiores pode ser difícil.

As conclusões positivas deste artigo são que existem classes específicas de conceitos que podem ser apre ndidos em tempo polinomial usando protocolos de aprendizagem do tipo descrito. Todas essas classes pode m ser caracterizadas definindo a classe de programas que as reconhecem. Em cada caso, os programas são tipos especiais de expressões booleanas. As três classes são (l) expressões de forma normal conjuntiva com um número limitado de literais em cada cláusula, (2) expressões monótonas de forma normal disjuntiva e (3) expressões arbitrárias nas quais cada variável ocorre apenas uma vez. No primeiro deles, não são necessárias chamadas do oráculo. No último caso, não é necessário acesso a exemplos típicos, mas os oráculos precisam ser mais poderosos do que o descrito acima.

O procedimento de dedução produzirá, em cada caso, uma expressão que, com alta probabilidade, se aproxime da expressão a ser aprendida. Uma tal expressão aproximada nunca diz sim quando não deveria, mas pode dizer não numa pequena fracção do espaço de probabilidade de exemplos positivos. Esta fração pode ser tornada arbitrariamente pequena aumentando o tempo de execução do procedimento de dedução. Talvez a principal descoberta técnica contida no artigo seja que com esta noção probabilística de aprendizagem uma aprendizagem altamente convergente é possível para classes inteiras de funções booleanas. Isto parece distinguir esta abordagem das abordagens mais tradicionais, onde a aprendizagem é vista como um processo de "indução" de alguma regra geral a partir de informação que é insuficiente para que seja feit a uma dedução fiável. O poder deste ponto de vista probabilístico é ilustrado, por exemplo, pelo facto de não se poder confiar num conjunto arbitrário de muitos exe mplos positivos polinomiais para determinar expressões que consistam mesmo num único monômio de qualquer forma fiável. Uma descrição mais detalhada deste problema metodológico pode ser encontrada em [9].

Há outro aspecto da nossa formulação que vale a pena enfatizar. Num sistema de aprendizagem que está prestes a aprender um novo conceito, pode haver um enorme número de variáveis proposicionais disponíveis. Podem ser entradas primitivas, valores de conceitos pré-programados ou valores de conceitos que foram desenvolvidos anteriormente. Queremos que a complexidade da aprendizagem do novo conceito esteja relacionada apenas com o número de variáveis que podem ser definidas em exemplos naturais do mesmo, e não com a cardinalidade do universo de variáveis disponíveis. Conseqüentemente, as perguntas feitas ao ORACLE e os valores dados por EXEMPLOS não serão atribuições de verdade para todas as variáveis. No caso arquetípico, eles especificarão uma atribuição a um subconjunto de variáveis que ainda é suficiente para garantir a verdade da função.

Se as classes de conceitos booleanos que podem ser aprendidos podem ser estendidas significativamente além das três classes fornecidas é uma questão interessante. Contudo, há evidências circunstanciais provenientes da criptografia de que toda a classe de funções computáveis por circuitos de tamanho polinomial não pode ser aprendida. Considere um esquema criptográfico que codifica mensagens avaliando a função Ek onde k es pecifica a chave. Suponha que este esquema seja imune ao ataque de texto simples escolhido no sentido de que, mesmo que os valores de Ek sejam conhecidos para muitas entradas polinomialmente escolhidas dinamicamente, é computacionalmente inviável deduzir um algoritmo para Ek ou para uma aproximação a ele. Isto equivale a dizer, entretanto, que Ek não pode ser aprendido. A suposta existência de boas funções criptográficas que são fáceis de calcular implica, portanto, que algumas funções fáceis de calcular não podem ser aprendidas.

Se a classe de conceitos que podem ser aprendidos for tão severamente limitada como sugerido pelos nossos resultados, então seguir-se-ia que a única maneira de ensinar conceitos mais complicados é construí-los a partir de conceitos tão simples. Assim, um bom professor teria que identificar, nomear e sequenciar esses conceitos intermediários à maneira de um programador. Os resultados da teoria da capacidade de aprendizagem indicariam então a granularidade máxima do único

concepts that can be acquired without programming.

In summary, this paper attempts to explore the limits of what is learnable as allowed by algorithmic complexity. The results are distinguishable from the diverse body of previous work on learning because they attempt to reconcile the three properties ((1)–(3)) mentioned earlier. Closest in rigor to our approach is the inductive inference literature (see Angluin and Smith [1] for a survey) that deals with inducing such things as recursive functions or formal grammars (but not Boolean functions) from examples. There is a large body of work on pattern recognition and classification, using statistical and other tools (e.g., [4]), but the question of general knowledge representation is not addressed there directly. Learning, in various less formal senses, has been studied widely as a branch of artificial intelligence. A survey and bibliography can be found in [2, 7]. In their terminology the subject of this paper is concept learning.

In previous work on learning, much emphasis is placed on the diversity of methods that humans appear to be using. These include learning by example, learning by analogy, and learning by being told. This paper emphasizes the polarity between learning by being programmed and learning in the absence of any programming element. Many of the conventionally accepted modes of learning can be regarded as being hybrids of these two extremes.

## 2. A LEARNING PROTOCOL FOR BOOLEAN FUNCTIONS

We consider $t$ Boolean variables $p_1, \ldots, p_t$, each of which can take the value 1 or 0 to indicate whether the associated proposition is true or false. There is no assumption about the independence of the variables. Indeed, they may be functions of each other.

A *vector* is an assignment to each of the $t$ variables of a value from $\{0, 1, *\}$. The symbol $*$ denotes that a variable is *undetermined*. A vector is *total* if every variable is *determined* (i.e., is assigned a value from $\{0, 1\}$). For example, the assignment $p_1 = p_3 = 1$, $p_4 = 0$, and $p_2 = *$ is a vector that is not total.

A Boolean function $F$ is a mapping from the set of $2^t$ total vectors to $\{0, 1\}$. A Boolean function $F$ becomes a *concept* $F$ if its domain is extended to the set of all vectors as follows: For a vector $v$, $F(v) = 1$ if and only if $F(w) = 1$ for all total vectors $w$ that agree with $v$ on all variables for which $v$ is determined. The purpose of this extension is that it permits us not to mention the variables on which $F$ does not depend.

Given a concept $F$, we consider an *arbitrary* probability distribution $D$ over the set of all vectors $v$ such that $F(v) = 1$. In other words, for each $v$ such that $F(v) = 1$, it is the case that $D(v) \geq 0$. Also $\sum D(v) = 1$ when summation is over this set of vectors. There are no other assumed restrictions on $D$, which is intended to describe the relative frequency with which the positive examples of $F$ occur in nature.

What are reasonable learning protocols to consider? First we must avoid giving the teacher too much power, namely, the power to communicate a program instruction by instruction. For example, if a premeditated sequence of vectors with repetitions could be communicated, then this could be used to encode the description of the program even if just two such vectors were used for binary notation. Second we must avoid giving the teacher what is evidently too little power. In particular, the protocol must provide some typical examples of vectors for which $F$ is true, for otherwise, if $F$ is true for just one vector that is total, only an exponential search or more powerful oracles would be able to find it.

Such considerations led us to consider the following learning protocol as a reasonable one. It gives the learner access to the following two routines:

1. EXAMPLES: This has no input. It gives as output a vector $v$ such that $F(v) = 1$. For each such $v$, the probability that $v$ is output on any single call is $D(v)$.
2. ORACLE( ): On vector $v$, as input it outputs 1 or 0 according to whether $F(v) = 1$ or 0.

The first of these gives randomly chosen positive examples of the concept being learned. The second provides a test for whether a vector, which the deduction procedure considers to be critical, is an example of the concept or not. In a real system, the oracle may be a human expert, a database of past observations, some deduction system, or a combination of these.

Finally we observe that our particular choice of learning protocol was strongly influenced by the earlier decision to deal with concepts rather than raw Boolean functions. If we were to deal with the latter, then many other alternatives are equally natural. For example, given a vector $v$, instead of asking, as we do, whether all completions of it make the function true, we could ask whether there exist any completions that make it true. This suggests natural alternative semantics for EXAMPLES and ORACLE. In Section 7 we shall use such more elaborate oracles.

## 3. LEARNABILITY

We shall consider various classes of programs having $p_1, \ldots, p_t$ as inputs and show that in each case any program in the class can be deduced, with only small probability of error, in polynomial time using the protocol previously described. We assume that programs can take the values 1, 0, and undetermined.

More precisely we say that a class $X$ of programs is *learnable* with respect to a given learning protocol if and only if there exists an algorithm $A$ (the deduction procedure) invoking the protocol with the following properties:

1. The algorithm runs in time polynomial in an adjustable parameter $h$, in the various parameters that quantify the size of the program to be learned, and in the number of variables $t$.
2. For all programs $f \in X$ and all distributions $D$ over vectors $v$ on which $f$ outputs 1, the algorithm will deduce with probability at least $(1 - h^{-1})$ a program $g \in X$ that never outputs one when it should not but outputs one almost always when it should. In





conceitos que podem ser adquiridos sem programação.

Em resumo, este artigo tenta explorar os limites do que pode ser aprendido conforme permitido pela complexidade algorítmica. Os resultados distinguem-se do conjunto diverso de trabalhos anteriores sobre aprendizagem porque tentam conciliar as três propriedades ((1)-(3)) mencionadas anteriormente. Mais próximo em rigor da nossa abordagem é a literatura de inferência indutiva (ver Angluin e Smith[1] para uma pesquisa) que trata da indução de coisas como funções recursivas ou gramáticas formais (mas não funções booleanas) a partir de exemplos. Há um grande volume de trabalhos sobre reconhecimento e classificação de padrões, usando ferramentas estatísticas e outras (por exemplo, [4]), mas a questão da representação do conhecimento geral não é abordada diretamente. A aprendizagem, em vários sentidos menos formais, tem sido amplamente estudada como um ramo da inteligência artificial. Uma pesquisa e bibliografia podem ser encontradas em[2, 7]. Na sua terminologia, o tema deste artigo é a aprendizagem de conceitos.

Em trabalhos anteriores sobre aprendizagem, muita ênfase é colocada na diversidade de métodos que os humanos parecem estar usando. Estas incluem aprender pelo exemplo, aprender por analogia e aprender por ouvir. Este artigo enfatiza a polaridade entre aprender sendo programado e aprender na ausência de qualquer elemento de programação. Muitos dos modos de aprendizagem convencionalmente aceites podem ser considerados híbridos destes dois extremos.

2. UM PROTOCOLO DE APRENDIZAGEM PARA FUNÇÕES BOOLEANAS
Consideramos t variáveis booleanas

por favor. . . . . pt, cada um dos quais pode assumir o valor 1 ou 0 para indicar se a proposição associada é verdadeira ou f alsa. Não há suposição sobre a independência das variáveis. Na verdade, eles podem ser funções um do outro.

Um vetor é uma atribuição a cada uma das t variáveis de um valor de {0, 1, *}. O símbolo * indica que uma variável é indeterminada. Um vetor é total se cada variável for determinada (ou seja, é atribuído um valor de {0, 1}). Por exemplo, a atribuição pl = p3 = 1, p4 = 0 e p2 = * é um vetor que não é total.

Uma função booleana F é um mapeamento do conjunto de 2 t vetores totais para {0, 1}. Uma função booleana F torna-se um conceito F se seu domínio for estendido ao conjunto de todos os vetores da seguinte forma: Para um vetor v, F(v) = 1 se e somente se F(w) = 1 para todos os vetores totais w que concordam com v em todas as variáveis para as quais v é determinado. O objetivo desta extensão é permitir-nos não mencionar as variáveis das quais F não depende.

Dado um conceito F, consideramos uma distribuição de probabilidade arbitrária D sobre o conjunto de todos os vetores v tal que F(v) = 1. Em outras palavras, para cada v tal que F(v) = 1, é o caso que D(v} _>O. Também ~ D(v) = 1 quando a soma é sobre este conjunto de vetores. Não há outras restrições assumidas em D, que se destina a descrever a frequência relativa com que os exemplos positivos de F ocorrem na natureza.

Quais são os protocolos de aprendizagem razoáveis a serem considerados? Primeiro, devemos evitar dar demasiado poder ao professor, nomeadamente, o poder de comunicar uma instrução de programa.

instrução. Por exemplo, se uma sequência premeditada de vetores com repetições pudesse ser comunicada, então isso poderia ser usado para codificar a descrição do programa mesmo que apenas dois desses vetores fossem usados para notação binária. Em segundo lugar, devemos evitar dar ao professor o que é evidentemente pouco poder. Em particular, o protocolo deve fornecer alguns exemplos típicos de vetores para os quais F é verdadeiro, caso contrário, se F for verdadeiro para apenas um vetor total, apenas uma busca exponencial ou oráculos mais poderosos seriam capazes de encontrá-lo.

Tais considerações nos levaram a considerar o seguinte protocolo de aprendizagem como razoável. Dá ao aluno acesso às duas rotinas a seguir:

1. EXEMPLOS: Não tem entrada. Fornece como saída um vetor v tal que F(v) = 1. Para cada um desses v, a probabilidade de v ser gerado em qualquer chamada única é D(v). 2. ORACLE( ): No vetor v, como entrada ele gera 1 ou 0 dependendo se F(v) = 1 ou 0.

O primeiro deles fornece exemplos positivos escolhidos aleatoriamente do conceito que está sendo aprendido. A segunda fornece um teste para saber se um vetor, que o procedimento de dedução considera crítico, é um exemplo do conceito ou não. Num sistema real, o oráculo pode ser um especialista humano, um banco de dados de observações passadas, algum sistema de dedução ou uma combinação destes.

Finalmente, observamos que a nossa escolha particular de protocolo de aprendizagem foi fortemente influenciada pela decisão anterior de lidar com conceitos em vez de funções booleanas brutas. Se tivéssemos de lidar com este último, então muitas outras alternativas seriam igualmente naturais. Por exemplo, dado um vetor v, em vez de perguntar, como fazemos, se todas as suas conclusões tor nam a função verdadeira, poderíamos perguntar se existe alguma conclusão que a torne verdadeira. Isto sugere uma semântica alternativa natural para EXAMPLES e ORACLE. Na Seção 7 usaremos esses oráculos mais elaborados.

3. APRENDIZAGEM Devemos considerar va

Várias classes de programas com pl . . . . . pt como entradas e mostram que em cada caso qualquer programa da classe pode ser deduzido, com apenas uma pequena probabilidade de erro, em tempo polinomial usando o protocolo descrito anteriormente. Assumimos que os programas podem assumir os valores 1, 0 e indeterminado.

Mais precisamente, dizemos que uma classe X de programas é aprendível em relação a um determinado protocolo de aprendizagem se e somente se existir um algoritmo A {o procedimento de dedução} que invoque o protocolo com as seguintes propriedades:

1. O algoritmo funciona em tempo polinomial num parâmetro ajustável h, nos vários parâmetros que quantificam o tamanho do programa a ser aprendido, e no número de variáveis t.

2. Para todos os programas f E X e todas as distribuições D sobre vetores v nos quais f produz 1, o algoritmo deduzirá com probabilidade pelo menos (1 - h -1} um programa g ~ X que nunca produz um quando não deveria, mas produz um quase sempre quando deveria. Em

particular, (i) for all vectors $v$, $g(v) = 1$ implies $f(v) = 1$, and (ii) the sum of $D(v)$ over all $v$ such that $f(v) = 1$, but $g(v) \neq 1$ is at most $h^{-1}$.

In our definition we have disallowed positive answers from $g$ to be wrong, only because the particular classes $X$ in this paper do not require it. This we call one-sided error learning. In other circumstances it may be efficacious to allow two-sided errors. In that more general case, we would need to put a probability distribution $\bar{D}$ on the set of vectors such that $f(v) \neq 1$. Condition (i) in (2) is then replaced by $\sum \bar{D}(v)$ over all $v$ such that $f(v) \neq 1$, but $g(v) = 1$ is at most $h^{-1}$.

A second aspect of our definition is that the parameter $h$ is used in two independent probabilistic bounds. This simplifies the statement of the results. It would be more precise, however, to work explicitly with two independent parameters $h_1$ and $h_2$.

Third we should note that programs that compute concepts should be distinguished from those that compute merely the Boolean function. This distinction has little consequence for the three classes of expressions that we consider, since in these cases programs for the concepts follow directly from the specification of the expressions. For the sake of generality, our definitions do allow the value of a program to be undefined, since a nontotal vector will not, in general, determine the value of an expression as 0 or 1.

It would be interesting to obtain negative results, other than the informal cryptographic evidence mentioned in the introduction, indicating that the class of unrestricted Boolean circuits is not learnable. A recent result [6] in this direction does establish this, under the hypothesis that integer factorization is intractable. Their result applies to the learning protocol consisting of EXAMPLES and ORACLE, when the latter is restricted to total vectors. It may also be interesting to go in the opposite direction and determine, for example, whether the hypothesis that $P$ equals $NP$ would imply that all interesting classes are learnable.

## 4. A COMBINATORIAL BOUND

The probabilistic analysis needed for our later results can be abstracted into a single lemma. The proof of the lemma is independent of the rest of the paper.

We define the function $L(h,s)$ for any real number $h$ greater than one and any positive integer $S$ as follows: Let $L(h,S)$ be the smallest integer such that in $L(h,S)$ independent Bernoulli trials each with probability at least $h^{-1}$ of success, the probability of having fewer than $S$ successes is less than $h^{-1}$.

The relevance of the function $L(h,S)$ to learning can be illustrated as follows. Consider an urn $U$ containing a very large number of marbles possibly of many different types. Suppose we wish to "learn" what kinds of marbles $U$ contains by taking a small random sample $X$. If all the marbles in $U$ are of distinct types, then it is clearly impossible to deduce much information about them. Suppose, however, that we know that there are only $S$ types of marbles and that we wish to choose $X$ so that it contains representatives of all but 1 percent of the marbles in $U$. Whatever the respective frequencies of the $S$ types may be, the definition of $L(h,S)$ implies that if $|X| = L(100,S)$ then with probability greater than 99 percent we will have succeeded. To see why this implication holds, suppose that after $L(100,S)$ successive choices marble types representing between them at least 1 percent of the urn remain unchosen. In that case we have made $L(100,S)$ Bernoulli trials each with probability at least 1 percent of success (i.e., of choosing a previously unchosen type) and have succeeded fewer than $S$ times (since some type remains unchosen). Note that what constitutes a success here will depend on previous choices. The probability of success, however, will always be at least 1 percent, independent of previous choices, and this is sufficient for our argument.

The following simple upper bound holds for the whole range of values of $S$ and $h$ and shows that $L(h,S)$ is essentially linear both in $h$ and in $S$.

PROPOSITION: *For all integers $S \geq 1$ and all real $h > 1$.*

$$L(h,S) \leq 2h(S + \log_e h).$$

**Proof**: We use the following three well-known inequalities of which the last is due to Chernoff (see [6] page 18).

1. For all $x > 0$, $(1 + x^{-1})^x < e$.
2. For all $x > 0$, $(1 - x^{-1})^x < e^{-1}$.
3. In $m$ independent trials, each with probability at least $p$ of success, the probability that there are at most $k$ success, where $k < mp$, is at most

$$\left(\frac{m - mp}{m - k}\right)^{m-k}\left(\frac{mp}{k}\right)^{k}.$$

The first factor in the expression in (3) above can be rewritten as

$$\left(1 - \frac{mp - k}{m - k}\right)^{((m-k)/(mp-k))(mp-k)}.$$

Using (2) with $x = (m - k)/(mp - k)$, we can upper bound the product by

$$e^{-mp+k}(mp/k)^k.$$

Substituting $m = 2h(S + \log_e h)$, $p = h^{-1}$, and $k = S$ and using logarithms to the base $e$ gives the bound

$$e^{-2S-2\log h+S}\cdot[2(1 + (\log h)/S)]^{(S/\log h)\log h}.$$

Rewriting this using (1) with $x = (\log_e h)/S$ gives

$$e^{-S-2\log h}\cdot 2^S\cdot e^{\log h} \leq (2/e)^S\cdot e^{-\log h} \leq h^{-1}.$$

As an illustration of an application, suppose that we have access to EXAMPLES, a source of natural positive examples of vectors for concept $F$. Suppose that we want to find an approximation to the subset $P$ of variables that are determined in at least one natural example of $F$. Consider the following procedure: Pick the first $L(h, |P|)$ vectors provided by EXAMPLES, and let $P'$ be the union of the sets of variables determined by these vectors. The definition then implies that with probability at least $1 - h^{-1}$ the set $P'$ will have the following





particular, (i) para todos os vetores v, g(v) = i implica f(v) = 1, e (ii) a soma de D(v) sobre todos v tal que f(v) = 1, mas g(v) ~ 1 é no máximo h-1.

Na nossa definição, não permitimos que respostas positivas de g estivessem erradas, apenas porque as classes X específicas neste artigo não o exigem. Chamamos isso de aprendizado de erro unilateral; em outras circunstâncias, pode ser eficaz permitir erros bilaterais. Nesse caso mais geral, precisaríamos colocar uma distribuição de probabilidade D no conjunto de vetores tal que f{v} ~ 1. A condição (i) in (2} é então substituída por ~ D{v} sobre t odo v tal que f(v) ~ 1, mas g{v) = 1 é no máximo h-1.

Um segundo aspecto da nossa definição é que o parâmetro h é usado em dois limites probabilísticos independentes. Isso simplifica a declaração dos resultados. Seria mais preciso, entretanto, trabalhar explicitamente com dois parâmetros independentes hi e h2.

Terceiro, devemos notar que os programas que computam conceitos devem ser distinguidos daqueles que computam apenas a função booleana. Esta distinção tem poucas consequências para as três classes de expressões que consideramos, uma vez que nestes casos os programas para os conceitos decorrem directamente da especificação das expressões. Por uma questão de generalidade, as nossas definições permitem que o valor de um programa seja indefinido, uma vez que um vector não total não determinará, em geral, o valor de uma expressão como 0 ou 1.

Seria interessante obter resultados negativos, além da evidência criptográfica informal mencionada na introdução, indicando que a classe de circuitos booleanos irrestritos não é aprendível. Um resultado recente [6] nessa direção estabelece isso, sob a hipótese de que a fatoração de inteiros é intratável.

Seu resultado se aplica ao protocolo de aprendizagem composto por EXEMPLOS e ORACLE, quando este último é restrito a vetores totais. Também pode ser interessante ir na direção oposta e determinar, por exemplo, se a hipótese de que P é igual a NP implicaria que todas as classes interessantes são aprendíveis.

4. UM LIMITE COMBINATORIAL A análise probabilística necessária para

ou nossos resultados posteriores podem ser resumidos em u m único lema. A prova do lema é independente do resto do artigo.

Definimos a função L(h,s) para qualquer número real h maior que um e qualquer número inteiro positivo S como segue: Seja L(h,S) o menor número inteiro tal que em L(h,S) testes de Bernoulli independentes, cada um com probabilidade de pelo menos h-1 de sucesso, a probabilidade de ter menos que S sucessos é menor que h -1.

A relevância da função L(h,S) para a aprendizagem pode ser ilustrada da seguinte forma: Considere uma urna U contendo um grande número de bolinhas de gude, possivelmente de muitos tipos diferentes. Suponha que desejamos "aprender" que tipos de bolinhas de gude U contém tomando uma pequena amostra aleatória X. Se todas as bolinhas de gude em U são de tipos distintos, então é claramente impossível deduzir muita informação sobre elas. Suponhamos, no entanto, que sabemos que existem apenas S tipos de bolinhas de gude e que desejamos escolher X de modo que contenha representantes de todos, exceto 1 por cento de bolinhas de gude.

as bolinhas de gude em U. Quaisquer que sejam as respectivas frequências dos tipos S, a definição de L(h,S} implica que se iX [ = L(100,S) então com probabilidade maior que 99 por cento teremos sucesso. Para ver por que essa implicação é válida, suponha que depois de L(100,S} escolhas sucessivas de tipos de bolinhas de gude representando entre eles pelo menos 1 por cento da urna permanecem sem escolha. Nesse caso fizemos L [100,S) testes de Bernoulli cada um com probabilidade de pelo menos 1 por cento de sucesso 0.e., de escolher um tipo previamente não escolhido} e ter tido sucesso menos de S vezes (uma vez que algum tipo permanece não escolhido}. Note que o que constitui um sucesso aqui dependerá de escolhas anteriores. A probabilidade de sucesso, no entanto, será s empre de pelo menos 1 por cento, independente de escolhas anteriores, e isto é suficiente para o nosso argumento.

O seguinte limite superior simples é válido para todo o intervalo de valores de S e h e mostra que L(h,S) é essencialmente linear tanto em h quanto em S.

PROPOSITION: *For all integers $S \geq 1$ and all real $h > 1$.*

$$L(h,S) \leq 2h(S + \log_e h).$$

Prova: Usamos as seguintes três desigualdades bem conhecidas, das quais a última se deve a Chernoff (ver [6] página 18).

1. Para a l l x > 0, (1 + x-l} x <e . 2.
Para a l l x >0, ( 1 - x - 1 } x <e -I.

3. Em m tentativas independentes, cada uma com probabilidade de pelo menos p de sucesso, a probabilidade de haver no máximo k sucesso, onde k < mp, é no máximo

$$\left(\frac{m - mp}{m - k}\right)^{m-k}\left(\frac{mp}{k}\right)^{k}.$$

O primeiro fator na expressão em (a) acima pode ser reescrito como

$$\left(1 - \frac{mp - k}{m - k}\right)^{((m-k)/(mp-k))(mp-k)}.$$

Usando (2) com x = (m - k)/(mp - k}, podemos li mitar superiormente o produto por

$$e^{-mp+k}(mp/k)^k.$$

Substituindo m = 2h(S + log~h), p = h -1 e k = S e usando logaritmos na base e fornece o limite

$$e^{-2S-2\log h+S}\cdot[2(1 + (\log h)/S)]^{(S/\log h)\log h}.$$

Reescrever isso usando (1} com x = (log~h}/S dá

$$e^{-S-2\log h}\cdot 2^S\cdot e^{\log h} \leq (2/e)^S\cdot e^{-\log h} \leq h^{-1}.$$

Como ilustração de uma aplicação, suponha que tenhamos acesso a EXEMPLOS, uma fonte de exemplos naturais positivos de vetores para o conceito F. Suponha que queremos encontrar uma aproximação para o subconjunto P de variáveis que são determinadas em pelo menos um exemplo natural de F. Considere o seguinte procedimento: Escolha os primeiros vetores L(h, IPI} fornecidos por EXEMPLOS, e seja P ' a união dos conjuntos de variáveis determinadas por esses vetores. A definição então implica que com probabilidade de pelo menos 1 - h -1 o conjunto P ' terá o seguinte

property: If a vector is chosen with probability distribution $D$, then the probability that it determines a variable in $P - P'$ is less than $h^{-1}$. The argument used above for the urn model establishes this as follows: If $P'$ does not have the claimed property, then the following has happened: $L(h, |P|)$ trials have been made (i.e., calls of EXAMPLES) each with probability greater than $h^{-1}$ of success (i.e., finding a vector that determines a variable in $P - P'$, but there have been fewer than $|P|$ successes (i.e., discoveries of new additions to $P'$). The probability of such a happening is less than $h^{-1}$ by the definition of $L$.

The above application shows that the set of variables that are determined in natural examples of $F$ can be approximated by a procedure whose runtime is independent of $t$, the total number of variables. It follows that in the definition of learnability the dependence on $t$ in part (1) can be relaxed to a dependence on the number of variables that are ever defined in positive examples.

## 5. BOUNDED CNF EXPRESSIONS

A *conjunctive normal form* (CNF) expression is any product $c_1c_2 \cdots c_r$ of clauses where each clause $c_i$ is a sum $q_1 + q_2 + \cdots + q_{j_i}$ of literals. A *literal* $q$ is either a variable $p$ or the negation $\bar{p}$ of a variable. For example, $(p_1 + \bar{p}_2)(\bar{p}_1 + \bar{p}_2 + p_3)$ is a CNF expression. For a positive integer $k$, a *k-CNF expression* is a CNF expression where each clause is the sum of at most $k$ literals.

For CNF expressions in general, we do not know of any polynomial time deduction procedure with respect to our learning protocol. The question of whether one exists is tantalizing because of its apparent simplicity.

In this section we shall prove that for any fixed integer $k$ the class of $k$-CNF expressions is learnable. In fact it is learnable easily in the sense that calls of EXAMPLES suffice and no calls of ORACLE are required.

In this and subsequent sections, we shall use the relation $\Rightarrow$ on concepts as follows: $F \Rightarrow G$ means that for all vectors $v$ whenever $F(v) = 1$ it is also the case that $G(v) = 1$. (N.B. This is equivalent to the relation $F \Rightarrow G$ when $F$, $G$ are regarded as functions.) For brevity we shall often denote both expressions and even vectors by the concepts they represent. Thus the concept represented by vector $w$ is true for exactly those vectors that agree with $w$ on all variables on which $w$ is determined. The concept represented by expression $f$ is obtained by considering the Boolean function computed by $f$ and extending its domain to become a concept. In this section we regard a clause $c_i$ to be a special kind of expression. In Sections 7 and 8, we consider monomials $m$, simple products of literals, as special forms of expressions. In all cases statements of the form $f \Rightarrow g$, $v \Rightarrow c_i$, $v \Rightarrow m$, or $m \Rightarrow f$ can be understood by interpreting the two sides as concepts or functions in the obvious way.

THEOREM A: *For any positive integer $k$, the class of $k$-CNF expressions is learnable via an algorithm A that uses $L = L(h, (2t)^{k+1})$ calls of EXAMPLES and no calls of ORACLE, where $t$ is the number of variables.*

**Proof**: The algorithm is initialized with formula $g$ as the product of all possible clauses of up to $k$ literals from $\{p_1, \bar{p}_1, p_2, \bar{p}_2, \ldots, p_t, \bar{p}_t\}$. Clearly the number of ways of choosing a clause of exactly $k$ literals is at most $(2t)^k$, and hence the number of ways of choosing a clause with up to $k$ literals is less than $2t + (2t)^2 + \cdots + (2t)^k < (2t)^{k+1}$. This bounds the initial number of clauses in $g$.

The algorithm then calls EXAMPLES $L$ times to give positive examples of the concept represented by $k$-CNF formula $f$. For each vector $v$ so obtained, it deletes all of the remaining clauses in $g$ that do not contain a literal that is determined to be true in $v$ (i.e., in the clauses deleted, each literal is either not determined or is negated in $v$). More precisely the following is repeated $L$ times:

**begin** $v :=$ EXAMPLES

for each $c_i$ in $g$ delete $c_i$ if $v \not\Rightarrow c_i$.

**end**

The claim is that with high probability the value of $g$ after the $L$th execution of this block will be the desired approximation to the formula $f$ that is being learned.

Initially the set of vectors $\{v \mid v \Rightarrow g\}$ is clearly empty. We first claim that as the algorithm proceeds the set $\{v \mid v \Rightarrow g\}$ will always be a subset of $\{v \mid v \Rightarrow f\}$. To prove this it is clearly sufficient to prove the same statement when both sets are restricted to only total vectors. Let $B$ be the product of all the clauses $c$ containing up to $k$ literals with the property that "$\forall v$ if $v \Rightarrow f$ then $v \Rightarrow c$." It is clear that in the course of the algorithm no clause in $B$ can ever be deleted. Hence it is certainly true that $\{v \mid v \Rightarrow g\} \subset \{v \mid v \Rightarrow B\}$. To establish the claim, it remains only to prove that $B$ computes the same Boolean function as $f$. (In fact $B$ will be the maximal $k$-CNF formula equivalent to $f$.) It is easy to see that $f \Rightarrow B$, since by the definition of $B$, for every $c$ in $B$ it is the case that "for all $v$ if $v \Rightarrow f$ then $v \Rightarrow c$." To verify the converse, that $B \Rightarrow f$, we first note that, by definition, $f$ is a $k$-CNF formula. If some clause in $f$ did not occur in $B$, then this clause $c'$ would have the property that "$\exists v$ such that $v \Rightarrow f$ but $v \not\Rightarrow c'$." But this is impossible since if $c'$ is a clause of $f$ and if $v \not\Rightarrow c'$ then $v \not\Rightarrow f$. We conclude that every $k$-CNF representation of the function that $f$ represents consists of a set of clauses that is a subset of the clauses in $B$. Hence $B \Rightarrow f$.

Let $X = \sum D(v)$ with summation over all (not necessarily total) vectors $v$ such that $v \Rightarrow f$ but $v \not\Rightarrow g$. This quantity is defined for each intermediate value of $g$ in the course of the algorithm and, as is easily verified, it is monotone decreasing with time. Now clauses will be removed from $g$ whenever EXAMPLES outputs a vector $v$ such that $v \not\Rightarrow g$. The probability of this happening at any moment is exactly the current value of X. Also, the process of running the algorithm to completion can have one of just two possible outcomes: (1) At some point $X$ becomes less than $h^{-1}$, in which case the $g$ found will approximate to $f$ exactly as required by the definition of learnability. (2) The value of $X$ never goes below $h^{-1}$, and hence $g$ is not an acceptable approximation. The probability of the latter eventuality is, how-





propriedade: Se um vetor for escolhido com distribuição de probabilidade D, então a probabilidade de ele determinar uma variável em P - P' é menor que h-1. O argumento usado acima para o modelo de urna estabelece isso da seguinte forma: Se P ' não tem a propriedade reivindicada, então aconteceu o seguinte: L(h, [P[) tentativas foram feitas (ou seja, chamadas de EXEMPLOS), cada uma com probabilidade maior que h-1 de sucesso (ou seja, encontrar um vetor que determina uma variável em P - P', mas houve menos de [P]sucessos (ou seja, descobertas de novas adições a P'). A probabilidade de tal acontecer é menor que h -1 pela definição de L.

A aplicação acima mostra que o conjunto de variáveis que são determinadas em exemplos naturais de F pode ser aproximado por um procedimento cujo tempo de execução é independente de t, o número total de variáveis. Segue-se que na definição de capacidade de aprendizagem a dependência de t na parte (1} pode ser relaxada para uma dependência do número de variáveis que são sempre definidas em exemplos positivos.

5. EXPRESSÕES CNF LIMITADAS Uma forma normal conjuntiva (CNF) expr

sessão é qualquer produto c~c2 ••• Cr de cláusulas onde cada cláusula ci é uma soma q~ + q2 + ••• + qi, de literais. Um literal q é uma variável p ou a negação p de uma variável. Por exemplo, (p~ + #2)(p~ + fi2 + pa) é uma expressão CNF. Para um número inteiro positivo k, uma expressão k-CNF é uma expressão CNF onde cada cláusula é a soma de no máximo k literais.

Para expressões CNF em geral, não conhecemos nenhum procedimento de dedução de tempo polinomial em relação ao nosso protocolo de aprendizagem. A questão de saber se existe um é tentadora devido à sua aparente simplicidade.

Nesta seção provaremos que para qualquer inteiro fixo k a classe de expressões k-CNF é aprendível. Na verdade, é facilmente aprendido no sentido de que chamadas de EXEMPLOS são suficientes e nenhuma chamada de ORACLE é necessária.

Nesta e nas seções subsequentes, usaremos a relação ~ sobre conceitos como segue: F ~ G significa que para todos os vetores v sempre que F(v) = 1 também é o caso que G(v} = i. (Observação: isso é equivalente à relação F ~ G quando F, G são considerados como funções.) Para ser breve, denotaremos frequentemente ambas as expressões e até mesmo vetores pelos conceitos que eles representam. Assim, o conceito representa. enviado pelo vetor w é verdadeiro exatamente para aqueles vetores que concordam com w em todas as variáveis nas quais w é determinado. O conceito representado pela expressão f é obtido considerando a função booleana calculada por f e estendendo seu domínio para se tornar um conceito. Nesta seção, consideramos uma cláusula cl como um tipo especial de expressão. a forma f ~ g, v ~ ci, v ~ m ou m ~ f pode ser entendida interpretando os dois lados como conceitos ou funções da maneira óbvia.

TEOREMA A: Para qualquer inteiro positivo k, a classe de expressões k-CNF pode ser aprendida através de um algoritmo A que usa L = L(h, (2t) k÷l) chamadas de EXEMPLOS e nenhuma chamada de ORACLE, onde t é o número de variáveis.

Prova: O algoritmo é inicializado com a fórmula g como o produto de todas as cláusulas possíveis de até k literais de { pl, pl, p2,/~2 . . . . . pt, #t}. Claramente, o número de maneiras de escolher uma cláusula com exatamente k literais é no máximo (2t} k e, portanto, o número de maneiras de escolher uma cláusula com até k literais é menor que 2t + (2t)2 + •••+ (2t) k< {2t} k+l. Isso limita o número inicial de cláusulas em g.

O algoritmo então chama EXEMPLOS L vezes para dar exemplos positivos do conceito representado pela fórmula k-CNF f. Para cada vetor v assim obtido, ele exclui todas as cláusulas restantes em g que não contêm um literal que é determinado como verdadeiro em v (ou seja, nas cláusulas excluídas, cada literal não é determinado ou é negado em v}. Mais precisamente, o seguinte é repetido L vezes:

começar v := EXEMPLOS

para cada ci em g exclua ci se v ~ cl.

A afirmação é que com alta probabilidade o valor de g após a L-ésima execução deste bloco será a aproximação desejada da fórmula f que está sendo aprendida.

Inicialmente o conjunto de vetores Iv[ v ~ g} está claramente vazio. Primeiro afirmamos que à medida que o algoritmo prossegue, o conjunto/v/v ~g } sempre será um subconjunto de {v Iv ~f } . Para provar isso é claramente suficiente provar a mesma afirmação quando ambos os conjuntos estão restritos apenas a vetores totais. Seja B o produto de todas as cláusulas c contendo até k literais com a propriedade "Vv se v ~ f então v ~ c". É claro que no decorrer do algoritmo nenhuma cláusula em B poderá ser excluída. Portanto, é certamente verdade que {v [ v ~ g} C {v/ v ~ B}. Para estabelecer a afirmação, resta apenas provar que B calcula a mesma função booleana que f. (Na verdade B será a fórmula k-CNF máxima equivalente a f.) é fácil ver que f~ B, já que pela definição de B, para cada c em B é o caso que "para todo v se v ~ f então v ~ c." Para verificar o inverso, que B ~ f, notamos primeiro que, por definição, f é uma fórmula k-CNF. Se alguma cláusula em f não ocorresse em B, então esta cláusula c' teria a propriedade que "3v tal que v ~f b u t v ~ c'." Mas isso é impossível, pois se c' é uma cláusula de f e se v ~~ c' então v ~ f. Concluímos que toda representação k-CNF da função que f representa consiste em um conjunto de cláusulas que é um subconjunto das cláusulas em B. Portanto, B ~ f.

Seja X = ~ D(v) com soma de todos os vetores (não necessariamente totais) v tais que v ~ f mas v ~ g. Esta quantidade é definida para cada valor intermediário de g no decorrer do algoritmo e, como é facilmente verificado, é monótona diminuindo com o tempo. Agora as cláusulas serão removidas de g sempre que EXAMPLES gerar um vetor v tal que v ~ g. A probabilidade de isso acontecer a qualquer momento é exatamente o valor atual de X. Além disso, o processo de execução do algoritmo até a conclusão pode ter um de apenas dois resultados possíveis: (1) Em algum ponto X torna-se menor que h -I, caso em que o g encontrado se aproximará de f exatamente como exigido pela definição de capacidade de aprendizagem. (2) O valor de X nunca fica abaixo de h -1 e, portanto, g não é uma aproximação aceitável. A probabilidade da última eventualidade é, no entanto

ever, at most $h^{-1}$ since it corresponds to the situation of performing $L(h, (2t)^{k+1})$ Bernoulli experiments (i.e., calls of EXAMPLES) each with probability greater than $h^{-1}$ of success (i.e., finding a $v$ such that $v \not\Rightarrow g$) and obtaining fewer than $(2t)^{k+1}$ successes (a success being manifested by the removal of at least one clause from $g$). □

In conclusion we observe that a CNF expression $g$ immediately yields a program for computing the associated concept. Given a vector $v$, we substitute the determined truth values in $g$. The concept will be true for $v$ if and only if all the clauses are made true by the substitution.

## 6. DNF EXPRESSIONS

A *disjunctive normal form* (DNF) expression is any sum $m_1 + m_2 + \cdots + m_r$ of monomials where each monomial $m_i$ is a product of literals. For example, $p_1\bar{p}_2 + p_1p_3\bar{p}_4$ is a DNF expression. Such expressions appear particularly easy for humans to comprehend. Hence we expect that any practical learning system would have to allow for them.

An expression is *monotone* if no variable is negated in it. We shall show that for monotone DNF expressions there exists a simple deduction procedure that uses both EXAMPLES and ORACLE. For unrestricted DNF a similar result can be proved with respect to a different size measure. In the unrestricted case, there is the additional difficulty that we can guarantee to deduce a program only for the function and not for the concept. This difficulty does not arise in the monotone case where given an expression we can always compute the value of the associated concept for a vector $v$ by making the substitution and asking whether the resulting expression is identically true.

A monomial $m$ is a *prime implicant* of a function $F$ (or of an expression representing the function) if $m \Rightarrow F$ and if $m' \not\Rightarrow F$ for any $m'$ obtained by deleting one literal from $m$. A DNF expression is *prime* if it consists of the sum of prime implicants none of which is redundant in the sense of being implied by the sum of the others. There is always a unique prime DNF expression in the monotone case, but not in general. We therefore define the *degree* of a DNF expression to be the largest number of monomials that a prime DNF expression equivalent to it can have. The unique prime DNF expression in the monotone case is simply the sum of all the prime implicants.

THEOREM B: *The class of monotone DNF expressions is learnable via an algorithm B that uses $L = L(h,d)$ calls of EXAMPLES and dt calls of ORACLE, where d is the degree of the DNF expression f to be learned and t the number of variables.*

**Proof:** The algorithm is initialized with formula $g$ identically equal to the constant zero. The algorithm then calls EXAMPLES $L$ times to produce positive examples of $f$. Each time a vector $v$ is produced such that $v \not\Rightarrow g$, a new monomial $m$ is added to $g$. The monomial $m$ is the product of those literals determined in $v$ that are essential to make $v \not\Rightarrow f$. More precisely, the loop that is executed $L$ times is the following:

```
begin v := EXAMPLES
    if v ⇏ g then
    begin for i := 1 to t do
        if p_i is determined in v then
        begin set ṽ equal to v but with p_i := *;
            if ORACLE(ṽ) = 1 then v := ṽ
        end
        set m equal to the product of all literals q
          such that v ⇒ q;
        g := g + m
    end
end
```

The test $v \not\Rightarrow g$ amounts to asking whether none of the monomials of $g$ are made true by the values determined to be true in $v$. Every time EXAMPLES produces a value $v$ such that $v \not\Rightarrow g$, the inner loop of the algorithm will find a prime implicant $m$ to add to $g$. Each such $m$ is different from any previously added (since the contrary would have implied that $v \Rightarrow g$). It follows that such a $v$ will be found at most $d$ times, and hence ORACLE will be called at most $dt$ times.

Let $X = \sum D(w)$ with summation over all (not necessarily total) vectors $w$ such that $w \Rightarrow f$ but $w \not\Rightarrow g$. This quantity is defined for each intermediate value of $g$ in the course of the algorithm, is initially unity, and decreases monotonically with time. Now a monomial will be added to $g$ each time EXAMPLES outputs a vector $v$ such that $v \not\Rightarrow g$. The probability of this occurring at any call of EXAMPLES is exactly X. The process of running the algorithm to completion can have just two possible outcomes: (1) At some time X has become less than $h^{-1}$, in which case the final expression $g$ found will approximate to $f$ as required by the definition of learnability; and (2) the value of X never goes below $h^{-1}$, and hence $g$ is not an acceptable approximation. The probability of this second eventuality is, however, at most $h^{-1}$, since it corresponds to the situation of performing $L(h,d)$ Bernoulli experiments (i.e., calls of EXAMPLES) each with probability greater than $h^{-1}$ of success (i.e., of finding a $v$ such that $v \Rightarrow f$ but $v \not\Rightarrow g$) and obtaining fewer than $d$ successes (each manifested by the addition of a new monomial). □

For unrestricted DNF expressions, several problems arise. The main source of difficulty is the fact that the problem of determining whether a nontotal vector implies the function specified by a DNF formula is *NP-hard*. This is simply because the problem of determining whether the nowhere determined vector implies the function is the tautology question of Cook [3]. An immediate consequence of this is that it is unreasonable to assume that the program being learned computes concepts rather than functions. Another consequence is that in any algorithm akin to Algorithm B the test $v \not\Rightarrow g$ may not be feasible if $v$ is not total. Algorithm B is sufficient, however, to establish the following:

THEOREM B′: *Suppose the notion of learnability is restricted to distributions D such that $D(v) = 0$ whenever v is not total. Then the class of DNF expressions is learnable, in*





sempre, no máximo h-1, pois corresponde à situação de realizar L(h, (2t) k+l) experimentos de Bernoulli (ou seja, chamadas de EXEMPLOS) cada um com probabilidade maior que h -1 de sucesso (ou seja, encontrar um v tal que v ~ g) e obter menos que (2t) k+l sucessos (um sucesso sendo manifestado pela remoção de pelo menos uma cláusula de g). []

Concluindo, observamos que uma expressão CNF g produz imediatamente um programa para calcular o conceito associado. Dado um vetor v, substituímos os valores verdade determinados em g. O conceito será verdadeiro para v se e somente se todas as cláusulas se tornarem verdadeiras pela substituição.

## 6. EXPRESSÕES DNF Uma forma normal disjuntiva

(DNF) expressão é qualquer soma ml + m2 + ••• + mr de monômios onde cada monômio mi é um produto de literais. Por exemplo, pl/~2+ plp3p4 é uma expressão DNF. Tais expressões parecem particularmente fáceis de serem compreendidas pelos humanos. Portanto, esperamos que qualquer sistema de aprendizagem prática tenha que permitir isso.

Uma expressão é monótona se nenhuma variável for negada nela. Mostraremos que para expressões DNF monótonas existe um procedimento de dedução simples que utiliza EXEMPLOS e ORACLE. Para o DNF irrestrito, um resultado semelhante pode ser provado com respeito a uma medida de tamanho diferente. No caso irrestrito, existe a dificuldade adicional que podemos garantir de deduzir um programa apenas para a função e não para o conceito. Esta dificuldade não surge no caso monótono em que dada uma expressão podemos sempre calcular o valor do conceito associado para um vector v fazendo a substituição e perguntando se a expressão resultante é identicamente verdadeira.

Um monômio m é um implicante principal de uma função F (ou de uma expressão que representa a função) se m ~ F e se m' ~,~ F para qualquer m' obtido pela exclusão de um literal de m. Uma expressão DNF é prima se consiste na soma de implicantes primos, nenhum dos quais é redundante no sentido de estar implícito na soma dos outros. Sempre há uma expressão DNF primária única no caso monótono, mas não em geral. Portanto, definimos o grau de uma expressão DNF como sendo o maior número de monômios que uma expressão DNF primária equivalente a ela pode ter. A expressão DNF primária única no caso monótono é simplesmente a soma de todos os implicantes primos.

TEOREMA B: A classe de expressões DNF monótonas pode ser aprendida por meio de um algoritmo B que usa chamadas L = L(h,d) de EXEMPLOS e chamadas dt do ORACLE, onde d é o grau da expressão DNF f a ser aprendida e t o número de variáveis.

Prova: O algoritmo é inicializado com a fórmula g idêntica à constante zero. O algoritmo então chama EXEMPLOS L vezes para produzir exemplos positivos de f. Cada vez que um vetor v é produzido tal que v ~ g, um novo monômio m é adicionado a g. O monômio m é o produto daqueles lit erais determinados em v que são essenciais para fazer v ~,~ f. Mais precisamente, o circuito que é executado L vezes é o seguinte:

começar v := EXEMPLOS se v ~ g então começar para i := 1 fazer se pi for determinado em v então começar definir ~ igual a v mas com pi := *; se ORACLE(W) = 1 então v :=end defina m igual ao produto de todos os literais q tal que v ~ q; g: =g +m fim fim

O teste v ~ g equivale a perguntar se nenhum dos monômios de g se torna verdadeiro pelos valores determinados como verdadeiros em v. Cada vez que EXAMPLES produz um valor v tal que v ~ g, o loop interno do algoritmo encontrará um implicante primo m para adicionar a g. Cada m é diferente de qualquer adicionado anteriormente (já que o contrário implicaria que v ~ g). Segue-se que tal v será encontrado no máximo d vezes e, portanto, ORACLE será chamado no máximo dt vezes.

Seja X = ~ D(w) com soma de todos os vetores (não necessariamente totais) w tais que w ~ f mas w ~ g. Esta quantidade é definida para cada valor intermediário de g no decorrer do algoritmo, é inicialmente unidade e diminui monotonicamente com o tempo. Agora, um monômio será adicionado a g cada vez que EXAMPLES gerar um vetor v tal que v ~~ g. A probabilidade de isso ocorrer em qualquer chamada de EXEMPLOS é exatamente X. O processo de execução do algoritmo até a conclusão pode ter apenas dois resultados possíveis: (1) Em algum momento X tornou-se menor que h -1, caso em que a expressão final g encontrada se aproximará de f conforme exigido pela definição de capacidade de aprendizagem; e (2) o valor de X nunca fica abaixo de h -1 e, portanto, g não é uma aproximação aceitável. A probabilidade desta segunda eventualidade é, no entanto, no máximo h -1, pois corresponde à situação de realizar L(h,d) experimentos de Bernoulli (ou seja, chamadas de EXEMPLOS} cada um com probabilidade maior que h-1 de sucesso (ou seja, de encontrar um v tal que v ~ f mas v ~ g) e obter menos que d sucessos (cada um manifestado pela adição de um novo monômio). []

Para expressões DNF irrestritas, surgem vários problemas. A principal fonte de dificuldade é o fato de que o problema de determinar se um vetor não total implica a função especificada por uma fórmula DNF é NP-difícil. Isto ocorre simplesmente porque o problema de determinar se o vetor determinado em lugar nenhum implica a função é a questão tautologia de Cook [3]. Uma consequência imediata disto é que não é razoável assumir que o programa que está a ser aprendido computa conceitos em vez de funções. Outra consequência é que em qualquer algoritmo semelhante ao Algoritmo B o teste v ~ g pode não ser viável se v não for total. O Algoritmo B é suficiente, entretanto, para estabelecer o seguinte:

TEOREMA B': Suponha que a noção de capacidade de aprendizagem seja restrita às distribuições D tais que D(v) = 0 sempre que v não for total. Então a classe de expressões DNF pode ser aprendida, em

*the sense that programs for the functions (but not necessarily the concepts) can be deduced in L(h,d) calls of EXAMPLES and dt calls of ORACLE where d is the degree of the DNF expression to be learned.*

The reader should note that here there is the additional philosophical difficulty that availability of the expression does not in itself imply a polynomial time algorithm for ORACLE. On the other hand, the theorem does say that if an agent has some, maybe ad hoc, black box for ORACLE whose workings are unknown to him or her, it can be used to teach someone else a DNF expression that approximates the function.

Finally, it may be worth emphasizing that the monotone case is nonproblematic and the deduction procedure for it very simpleminded. It may be interesting to pursue more sophisticated deduction procedures for it if contexts can be found in which they can be proved advantageous. The question as to whether monotone DNF expressions can be learned from EXAMPLES alone is open. A positive answer would be especially significant. A partial result in this direction can be obtained by dualizing Theorem A: For any $k$ the class of DNF expressions having a bound $k$ on the length of each disjunct can be learned in polynomial time from negative examples alone (i.e., from a source of vectors $v$ such that $\bar{D}(v) \neq 0$).

## 7. $\mu$-EXPRESSIONS

We have already seen a class of expressions, namely the $k$-CNF expressions, that can be learned from positive examples alone. Here we shall consider the other extreme, a class that can be learned using oracles alone.

Deducing expressions on which there are less severe structural restrictions than DNF or CNF appears much more difficult. The aim of this section is to give a paradigmatic example of how far one can go in that direction if one is willing to pay the price of oracles that are more sophisticated than the one previously described.

A general *expression* over variable set $\{p_1, \ldots, p_t\}$ is defined recursively as follows:

1. For each $i$ $(1 \leq i \leq t)$ "$p_i$" and "$\bar{p}_i$" are expressions;
2. if $f_1, \ldots, f_r$ are expressions, then "$(f_1 + f_2 + \cdots + f_r)$" is an expression (called a plus expression);
3. if $f_1, \ldots, f_r$ are expressions, then "$(f_1 \times f_2 \times \cdots \times f_r)$" is an expression (called a times expression).

A *$\mu$-expression* is an expression in which each variable appears at most once. We can assume that a $\mu$-expression is monotone, since we can always relabel negated variables with new names that denote their negation. In the recursive definition of any $\mu$-expression, there are clearly at most $2t - 1$ intermediate expressions of which at most $t$ are of type (1) and at most $t - 1$ of types (2) or (3). Without loss of generality, we shall assume that in the definition of a $\mu$-expression rules (2) and (3) alternate. We shall regard two $\mu$-expressions as identical if their definitions can be made formally the same by reordering sums and products and relabeling as necessary.

For learning $\mu$-expressions, we shall employ more powerful oracles. The boundary between reasonable and unreasonable oracles does not appear sharp. We make no claims about the reasonableness of these new oracles except that they may serve as vehicles for understanding learnability.

The definitions refer to the Boolean function $F$ (not regarded as a concept here). The oracle of Section 2 will be renamed $N$-ORACLE, since it is one of *necessity*: $N$-ORACLE$(v) = 1$ if and only if for all total vectors $w$ such that $w \Longrightarrow v$ it is the case that $F(w) = 1$. The dual of this would be a *possibility oracle*: $P$-ORACLE$(v) = 1$ if and only if there exists a total vector $w$ such that $w \Longrightarrow v$ and $F(w) = 1$. For brevity we shall also define the *prime implicant* oracle: $PI(v) = 1$ if and only if $N$-ORACLE$(v) = 1$, but $N$-ORACLE$(w) = 0$ for any $w$ obtained from $v$ by making one variable determined in $v$ undetermined.

Finally we define two further oracles, ones of *relevant possibility RP* and of *relevant accompaniment RA*. For convenience we define the first by how it behaves on vectors represented as monomials: $RP(m) = 1$ iff from some monomial $m'$ $mm'$ is a prime implicant of $f$. For sets $V, W$ of variables, we define $RA(V, W) = 1$ iff every prime implicant of $f$ that contains a variable from $V$ also contains a variable from $W$.

THEOREM C: *The class of $\mu$-expressions in learnable via a deduction procedure C that uses $0(t^3)$ calls of N-ORACLE RP and RA altogether, where t is the number of variables (and no calls of EXAMPLES). The procedure always deduces exactly the correct expression.*

**Proof:** Let $\lambda$ be the monomial that is the product of no literals. The algorithm will first compute $RP(p_i)$ for each $p_i$ to determine which of the variables occur in the prime implicants of the function that the expression $f$ to be learned represents. Let $g_1, \ldots, g_r$ be distinct single variable expressions, one for each $p_i$ for which $RP(p_i) = 1$. Note that these are exactly the variables that occur in $f$, since $f$ is monotone.

With each $g_i$ we associate two monomials $m_i$ and $\hat{m}_i$. The former will be the single variable $p_i$ that $g_i$ represents. The latter is defined as any monomial $\hat{m}$ having no variable in common with $m_i$ such that $m_i\hat{m}$ is a prime implicant of $f$. The algorithm will construct each such $\hat{m}_i$ as the final value of $m$ in the following procedure: Set $m = \lambda$; while $PI(mm_i) = 0$ find a $p_k$ not in $mm_i$ such that $RP(p_k m m_i) = 1$ and set $m := p_k m$. Hence in at most $t^2$ calls of $PI$ (i.e., $t^3$ calls of $N$-ORACLE) and $t^2$ calls of $RP$ values for every $\hat{m}_i$ will be found.

Once initialized the algorithm proceeds by alternately executing a *plus-phase* and a *times-phase*. At the start of each phase, we have a set of expressions $g_1, \ldots, g_r$ where each $g_i$ is associated with two monomials $m_i$ and $\hat{m}_i$ (having no variables in common) where $m_i$ is a prime implicant of $g_i$ and $m_i\hat{m}_i$ is a prime implicant of $f$. We distinguish the $g$s as *plus* or *times expressions* according to whether the outermost construction rule was addition or multiplication. A plus-phase will first compute an equivalence relation $S$ on the subset of $\{g_i\}$ that are times expressions. For each equivalence class $G$ such that no $g_i \in G$ already occurs as a summand in a





o sentido de que programas para as funções (mas não necessariamente os conceitos) podem ser deduzidos em chamadas L(h,d) de EXAMPLES e chamadas dt de ORACLE onde d é o grau da expressão DNF a ser aprendida.

O leitor deve notar que aqui existe a dificuldade filosófica adicional de que a disponibilidade da expressão não implica por si só um algoritmo de tempo polinomial para ORACLE. Por outro lado, o teorema diz que se um agente te m alguma caixa preta, talvez ad hoc, para ORACLE cujo funcionamento é desconhecido para ele ou ela, ela pode ser usada para ensinar a outra pessoa uma expressão DNF que se aproxima da função.

Finalmente, pode valer a pena enfatizar que o caso monótono não é problemático e o procedimento de dedução para ele é muito simples. Pode ser interessante prosseguir procedimentos de dedução mais sofisticados se puderem ser encontrados contextos em que possam ser provados vantajosos. A questão se as expressões DNF monótonas podem ser aprendidas apenas a partir de EXEMPLOS está em aberto. Uma resposta positiva seria especialmente significativa. Um resultado parcial nesta direção pode ser obtido pela dualização do Teorema A: Para qualquer k a classe de expressões DNF tendo um limite k no comprimento de cada disjunto pode ser aprendida em tempo polinomial apenas a partir de exemplos negativos (ou seja, de uma fonte de vetores v tal que /3(v) # 0).

7. a-EXPRESSÕES Já vi mos

n uma classe de expressões, nomeadamente as expressões k-CNF, que podem ser aprendidas apenas a partir de exe mplos positivos. Aqui consideraremos o outro extremo, uma classe que pode ser aprendida apenas usando oráculos.

Deduzir expressões nas quais existem restrições estruturais menos severas do que DNF ou CNF parece muito mais difícil. O objectivo desta secção é dar um exemplo paradigmático de até onde se pode ir nessa direcção se estivermos dispostos a pagar o preço de oráculos mais sofisticados do que o descrito anteriormente.

Uma expressão geral sobre o conjunto de variáveis {pl ..... pt} é definida recursivamente da seguinte forma:

1. Para cada i (1 _<i _<t) "pl" e "Pi" são expressões; 2. seff~ . . . . fr são expressões, então "(fl +f2 + .-" +fr)" é uma expressão (chamada de expressão positiva); 3. se fl . . . . . fi são expressões, então "(fl x f2 x . . . x fr)" é uma expressão (chamada de expressão de tempos).

Uma expressão a é uma expressão na qual cada variável aparece no máximo uma vez. Podemos assumir que uma expressão r é monótona, uma vez que sempre podemos renomear variáveis negadas com novos nomes que denotam sua negação. Na definição recursiva de qualquer expressão j u, existem claramente no máximo 2t - 1 expressões intermediárias das quais no máximo t são do tipo (1) e no máximo t - 1 dos tipos (2) ou (3). Sem perda de generalidade, assumiremos que na definição de uma expressão t as regras (2) e (3) se alternam. Consideraremos duas expressões/z co mo idênticas se as suas definições puderem ser formalmente idênticas, reordenando somas e produtos e reetiquetando-as conforme necessário.

Para aprender expressões a, empregaremos oráculos mais poderosos. A fronteira entre oráculos razoáveis e irracionais não parece nítida. Não fazemos nenhuma afirmação sobre a razoabilidade destes novos oráculos, exceto que eles podem servir como veículos para a compreensão da capacidade de aprendizagem.

As definições referem-se à função booleana F (não considerada aqui como um conceito). O oráculo da Seção 2 será renomeado como N-ORACLE, pois é necessário: N-ORACLE(v) = 1 se e somente se para todos os vetores totais w tais que w ~ v é o caso que F(w) = 1. O dual disso seria um oráculo de possibilidade: P-ORACLE(v) = 1 se e somente se existir um vetor total w tal que w ~ v e F(w} = 1. Para resumir, também definiremos o oráculo implicante principal: Pl(v) = 1 se e somente se N-ORACLE(v) =1, mas N-ORACLE(w) = 0 para qualquer w obtido de v fazendo um variável determinada em v indeterminada.

Finalmente definimos mais dois oráculos, aqueles de possibilidade relevante RP e de acompanhamento relevante RA. Por conveniência, definimos o primeiro pela forma como ele se comporta em vetores representados como monômios: RP(m) = 1 iff de algum monômio m' ram' é um implicante primo off. Para conjuntos V, W de variáveis, definimos RA(V, W) = 1 se cada implicante primo de f que contém uma variável de V também contém uma variável de W.

TEOREMA C: A classe de expressões in pode ser aprendida por meio de um procedimento de dedução C que usa O(t 3) chamadas de N-ORACLE RP e RA juntas, onde t é o número de variáveis (e nenhuma chamada de EXEMPLOS). O procedimento sempre deduz exatamente a expressão correta.

Prova: Seja ~ o monômio que não é produto de literais. O algoritmo irá primeiro calcular RP(pj) para cada pj para determinar quais das variáveis ocorrem nos implicantes primos da função que a expressão f a ser aprendida representa. Deixe gl. . . . . gr sejam expressões distintas de variável única, uma para cada pj para o qual RP(p/) =1. Observe que essas são exatamente as variáveis que ocorrem em f, já que f é monótona.

A cada gi associamos dois monômios mi e rhi. A primeira será a única variável pj que gi representa. Este último é definido como qualquer monômio rh que não tem nenhuma variável em comum com m~ tal que m~rh é um implicante primo off. O algoritmo construirá cada rhi como o valor final de m no seguinte procedimento: Defina m = ~; enquanto PI(mmi) = 0 encontre um p~ não em mini tal que RP(pkmmi) = 1 e defina m := pkm. Portanto, em no máximo t 2 chamadas de PI (ou seja, t 3 chamadas de N-ORACLE) e t 2 chamadas de valores de RP para cada rhi serão encontradas.

Uma vez inicializado, o algoritmo prossegue executando alternadamente uma fase positiva e uma fase temporal. No início de cada fase, temos um conjunto de expressões gl . . . . . gr onde cada gi está associado a dois monômios mi e rhi (não tendo variáveis em comum) onde mi é um implicante principal de gi e mirhi é um implicante principal de off. Distinguimos os gs como expressões de mais ou de vezes, dependendo se a regra de construção mais externa foi adição ou multiplicação. Uma fase positiva calculará primeiro uma relação de equivalência S no subconjunto de {gi} que são expressões vezes. Para cada classe de equivalência G tal que nenhum gi E G já ocorre como uma soma em um

sum, we construct a new expression that is the sum of the members of $G$ and call this sum $g_k$ where $k$ is a previously unused index. If some members of $G$ already occur in a sum, say $g_j$ (N.B. they are never distributed in more than one sum), then we modify the sum expression $g_j$ to equal the sum of every expression in $G$. A times-phase is exactly analogous except that it computes a different equivalence relation $T$, now on plus expressions, and will form new, or extend old, times expressions. In the above context, single variable expressions will be regarded as both times and plus expressions. Also, it is immaterial which of the two kinds of phase is used to start the algorithm.

The intention of the algorithm is best expressed by the claim to follow, which will be verified later. Suppose that the expressions occurring in the definition of $f$ are $f_1, \ldots, f_q$ (where $q \leq 2t - 1$). We shall say that $g \leq f_i$ iff the set of prime implicants of $g$ is a subset of the set of prime implicants of $\tilde{f}_i$ where (1) if $f_i$ is a plus expression then $\tilde{f}_i = f_i$ and (2) if $f_i$ is a times expression then $\tilde{f}_i$ is the product of some subset of the multiplicands of $f_i$.

**Claim 1:** After every phase of the algorithm for every $g_i$ that has been constructed, there is exactly one expression $f_i$ such that (1) $g_i \leq f_i$ and (2) $f_i$ is of the same kind (i.e., plus or times) as $g_i$.

The procedure builds up the rooted tree of the expression as rooted subtrees starting from the leaves. One evident difficulty is that there is no a priori knowledge available about the shape of the tree. Hence in the grafting process a subtree may become attached to another at just about any node.

Whenever a plus or times expression $g_i$ is created or enlarged, its associated $m_i$ and $\hat{m}_i$ are updated as follows. If $g_i$ is a sum, then we let $m_i = m_j$ and $\hat{m}_i = \hat{m}_j$ for any $g_j$ that is a summand in $g_i$. If $g_i$ is a product, then $m_i$ will be the product of all the $m_j$s that correspond to multiplicands in $g_i$. Finally $\hat{m}_i$ will be generated as the final value of $m$ in the following procedure: Set $m := \lambda$; while $PI(mm_i) = 0$ find a $p_k$ not in $mm_i$ such that $RP(p_k m m_i) = 1$ and set $m := p_k m$. Since such an $\hat{m}_i$ has to be found at most $t$ times in the overall algorithm, the total cost is at most $t^2$ calls of $PI$ (i.e., $t^3$ calls of N-ORACLE) and $t^2$ calls of $RP$.

In order to complete the description of the algorithm, it remains only to define the equivalence relations $S$ and $T$.

**Definition:** $g_i S g_j$ if and only if

1. $PI(m_i\hat{m}_j) = PI(m_j\hat{m}_i) = 1$ and
2. $m_i$, $\hat{m}_j$ contain disjoint sets of variables, as do $m_j$, $\hat{m}_i$.

For defining $T$ we shall denote by $V_i$ the set of variables that occur in the expression $g_i$.

**Definition:** $g_i T g_j$ if and only if $RA(V_i, V_j) = RA(V_j, V_i) = 1$.

First we shall verify that $S$ and $T$ are indeed equivalence relations under the assumption that Claim 1 holds at the beginning of each phase. Clearly $S$ and $T$ are defined to be both reflexive and symmetric. To verify that $T$ is transitive, suppose that $g_i T g_j$ and $g_j T g_k$. The former implies that every prime implicant of $f$ containing some variable from $g_i$ also contains some variable from $g_j$. The latter implies that every prime implicant of $f$ containing some variable from $g_j$ also contains a variable from $g_k$. Hence $g_i T g_k$ follows. In order to verify the transitivity of $S$, we shall make a more general observation about any subexpression of $f$.

**Claim 2:** If $m_i$, $m_j$ are prime implicants of distinct times subexpressions $f_i$, $f_j$ of $f$ and if for some $m$, with no variable in common with either $m_i$ or $m_j$, both $m_i m$ and $m_j m$ are prime implicants of $f$, then $f_i$, $f_j$ must occur as summands in the same plus expression of $f$.

**Proof:** Most easily verified by representing expression $f$ as a directed graph (e.g., [8]) with edges labeled by the Boolean variables. The sets of labels along directed paths between a distinguished source node and a distinguished sink node correspond exactly to prime implicants in the case of $\mu$ expressions where all edges will have different labels.

Now to verify that $S$ is transitive, suppose that $g_i S g_j$ and $g_j S g_k$ where, by Claim 1, $g_i \leq f_i$, $g_j \leq f_j$, and $g_k \leq f_k$ for appropriate times expressions $f_i$, $f_j$, $f_k$. Then it follows that $m_i\hat{m}_j$, $m_j\hat{m}_j$, and $m_k\hat{m}_j$ are all prime implicants of $f$ where $m_i$, $m_j$, $m_k$ have no variable in common with $\hat{m}_j$. It follows from Claim 2 that $f_i$, $f_j$, and $f_k$ are addends in the same sum expression. Hence $g_i S g_k$ follows.

**Claim 3:** Suppose that after some phase of the algorithm Claim 1 holds and times expressions $g_i$, $g_j$ have been formed where $g_i \leq f_i$, $g_j \leq f_j$, and $f_i$, $f_j$ are times expressions. Then $g_i$ and $g_j$ will be placed in the same sum expression at the next plus-phase if and only if $f_i = \tilde{f}_i$, $f_j = \tilde{f}_j$, and $f_i$, $f_j$ are addends in the same sum expression in $f$.

**Proof:** ($\Rightarrow$) If $g_i \leq f_i = \tilde{f}_i$, $g_j \leq f_j = \tilde{f}_j$, and $f_i$, $f_j$ are in the same sum, then $m_i$, $m_j$ will be prime implicants of $f_i$, $f_j$. Also $m_i\hat{m}_j$ and $m_j\hat{m}_i$ will be prime implicants of $f$, and the variables in $m_i$ will be disjoint from those in $\hat{m}_j$ as will be those in $m_j$ from those in $\hat{m}_i$. Hence $g_i S g_j$ will hold and $g_i, g_j$ will be in the same plus expression after the next plus-phase.

($\Leftarrow$) Suppose that $g_i \leq f_i$, $g_j \leq f_j$, and $g_i S g_j$ holds. Then $m_i$, $m_j$ will be prime implicants of $g_i$, $g_j$, respectively, $m_i\hat{m}_j$, $m_j\hat{m}_i$ will both be prime implicants of $f$, and the variables in $\hat{m}_j$ will be disjoint from those of both $m_i$ and $m_j$. It follows from Claim 2 that $f_i = \tilde{f}_i$ and $f_j = \tilde{f}_j$ must occur as summands in the same plus expression of $f$. [N.B. Here we are using the fact that Claim 2 remains true if we allow a "times subexpression" to be the product of any subset of the multiplicands in a times expression in $f$.]

**Claim 4:** Suppose that after some phase of the algorithm, Claim 1 holds and some plus expressions $g_i \leq f_i$ and $g_j \leq f_j$ ($f_i$, $f_j$ plus expressions) have been found. (1) If $g_i = f_i$, $g_j = f_j$, and $f_i$, $f_j$ are multiplicands in the same times expression in $f$, then $g_i$, $g_j$ will be placed in the same times expression after the next times phase. (2) If





soma, construímos uma nova expressão que é a soma dos membros de G e chamamos essa soma de gk onde k é um índice não utilizado anteriormente. Se alguns membros de G já ocorrem em uma soma, digamos gj (Observação: eles nunca são distribuídos em mais de uma soma), então modificamos a expressão de soma & para igualar a soma de cada expressão em G. Uma fase temporal é exatamente análoga, exceto que ela calcula uma relação de equivalência diferente T, agora em expressões positivas, e formará expressões de tempo novas ou antigas. No contexto acima, as expressões de variável única serão consideradas expressões de tempo e de sinal positivo. Além disso, não importa qual dos dois tipos de fase é usado para iniciar o algoritmo.

A intenção do algoritmo é melhor expressa pela afirmação a seguir, que será verificada posteriormente. Suponha que as expressões que ocorrem na definição de off sejam f~ . . . . . fq (onde q _<2t - 1}. Diremos que g _<fi se o conjunto de implicantes primos de g é um subconjunto do conjunto de implicantes primos de fi onde (1) iffi é uma expressão positiva então fi = fi e (2) iffi é uma expressão vezes então f~é o produto de algum subconjunto dos multiplicandos off~.

Afirmação 1: Após cada fase do algoritmo para cada g~ que foi construído, existe exatamente uma expressão fi tal que (1) gi -<fi e (2) fi é do mesmo tipo (ou seja, mais ou vezes) que gi.

O procedimento constrói a árvore enraizada da expressão como subárvores enraizadas começando nas folhas. Uma dificuldade evidente é que não existe conhecimento a priori disponível sobre a forma da árvore. Portanto, no processo de enxerto, uma subárvore pode ser anexada a outra em praticamente qualquer nó.

Sempre que uma expressão positiva ou multiplicada gi é criada ou ampliada, seus associados mi e thi são atualizados como segue. Se gi é uma soma, então consideramos mi = mj e rhi = ~j para qualquer & que seja uma soma em g~. Seg~ é um produto, então m~será o produto de todos os mjs que correspondem a multiplicandos em gi. Finalmente rhi será gerado como o valor final de m no seguinte procedimento: Set m := ~; enquanto Pl(mmi) = 0 encontre um pk que não esteja em mini tal que RP(pkmmi) = 1 e defina m := pkm. Como tal rhi deve ser encontrado no máximo t vezes no algoritmo geral, o custo total é no máximo t 2 chamadas de PI (ou seja, t 3 chamadas de N-ORACLE) e t 2 chamadas de RP.

Para completar a descrição do algoritmo, resta apenas definir as relações de equivalência S e T.

Definição: giS& se e somente se

1. PI(rnirhj) = PI(mjrhi) = 1 e 2. mi, ~j contêm conjuntos disjuntos de variáveis, assim como mj, mi.

Para definir T denotaremos por Vi o conjunto de variáveis que ocorrem na expressão gi.

Definição: giTgj se e somente se RA(Vi, Vj) = RA(V# Vi)

Primeiro verificaremos que S e T são de facto relações de equivalência sob a suposição de que a Afirmação 1 é válida no início de cada fase. Claramente S e T são definidos como reflexivos e simétricos. Para verificar se T é transitivo, suponha que giTgj e &Tgk. A primeira implica que todo implicante primo de f contendo alguma variável de gi também contém alguma variável de &. O último implica que todo implicante primo off contendo alguma variável de & também contém uma variável de gk. Daí giTgk segue. Para verificar a transitividade de S, faremos uma observação mais geral sobre qualquer subexpressão off.

Afirmação 2: Se mi, mj são implicantes primos de subexpressões de tempos distintos fi, fj o ff e se para algum m, sem nenhuma variável em comum com mi ou mj, tanto mira quanto mjm são implicantes primos off, então fi, fj devem ocorrer como somas na mesma expressão positiva off.

Prova: É mais facilmente verificada representando a expressão f como um gráfico direcionado (por exemplo, [8]) com arestas rotuladas pelas variáveis booleanas. Os conjuntos de rótulos ao longo de caminhos direcionados entre um nó fonte distinto e um nó coletor distinto correspondem exatamente aos implicantes primos no caso de # expressões onde todas as arestas terão rótulos diferentes.

Agora, para verificar se S é transitivo, suponha que g~S& e gjSgk onde, pela reivindicação 1, gi -<fi, & -<f# e gk ~ fk para expressões de tempos apropriados fi, fj, fk. Segue-se então que mini, mjrhj e mkrhj são todos implicantes primos de onde mi, m# mk não têm nenhuma variável em comum com rhj. Segue-se da reivindicação 2 que fi, fj e fk são adendas na mesma expressão de soma. Daí giSgk segue.

Afirmação 3: Suponha que após alguma fase do algoritmo a Afirmação 1 mantém e as expressões de tempos g~, gj fora m formadas onde gi -<fi, & -<~, e fi, fj são expressões de tempos. Então gi e & serão colocados na mesma expressão de soma na próxima fase positiva se e somente iffi =fi, fj = )~, e ffi, fj forem adendas na mesma expressão de soma em f.

Prova: (~) Se gi -<fi = fi, & <- fj -- )~j, e fi, fj estão na mesma soma, então mi, mj serão implicantes primos de „ ~. Além disso, rnithj e mjrhi serão implicantes primos, e as variáveis em mi serão disjuntas daquelas em rhj, assim como o serão aquelas em mj daquelas em rhi. Conseqüentemente, giSgj será mantido e gi,gj estará na mesma expressão positiva após a próxima fase positiva.

(~) Suponha que gi -<fi, gj ~fj e giSgj sejam válidos. Então mi, rnj serão implicantes primos de gi, &, respectivamente, m~ rhj, rnffhj serão ambos implicantes primos off, e as variáveis em rhj serão disjuntas daquelas de mi e mj. Segue-se da reivindicação 2 que fi = 1~e fj = )~j devem ocorrer como somas na mesma expressão positiva off. [Observação Aqui estamos usando o fato de que a afirmação 2 permanece verdadeira se permitirmos que uma "subexpressão vezes" seja o produto de qualquer subconjunto dos multiplicandos em uma expressão vezes em f.]

Afirmação 4: Suponha que após alguma fase do algoritmo, a Afirmação 1 seja válida e algumas expressões positivas gi -<f~e gj _<fj ~fi, fj mais expressões) tenham sido encontradas. (1) Se gi = fi, & = ~, e fi, fi são multiplicandos na mesma expressão de tempos em f, então &, & serão colocados na mesma expressão de tempos após a próxima fase de tempos. (2} Se

$g_i$, $g_j$ are placed in the same times expression at any subsequent phase, then $f_i$, $f_j$ are in the same product in $f$.

**Proof:** (1) If the conditions of (1) above hold, then $g_iTg_j$ will be discovered at the next times-phase and the claim follows. (2) If $f_i$, $f_j$ are not in the same product in $f$, then $f$ contains some prime implicant containing variables from one of $g_i$, $g_j$ and not from the other. Hence $g_iTg_j$ will never hold.

**Proof of Claim 1:** By induction on the number of phases on Claims 1, 3, and 4 simultaneously.

We define the *depth* of a formula $f_i$ to be the maximum number of alternations of sum and product required in its definition.

**Claim 5:** If $f_i$ is an expression of depth $k$ in $f$, then after $k$ phases a $g_i$ identical to $f_i$ will have been constructed by the algorithm.

**Proof:** By induction on Claims 3 and 4.

To conclude the proof of the theorem, it remains to analyze the runtime. This is dominated by the cost of computing $m_i$ and $\hat{m}_i$, for which we have already accounted, plus the cost of computing the equivalence relations $S$ and $T$. We first note that fewer than $2t$ expression names $g_i$ are used in the course of the algorithm. When an expression is grafted into another at a point deep in the latter, then the semantics of all the subexpressions above will change. By Claims 3 and 4 such grafting can occur when a $g_i$ is added to a sum expression but not when added to a times expression. Hence the values $m_i$ and $\hat{m}_i$ do not need to be changed for any expression when such grafting occurs. It follows that for computing $S$ only $2t$ values of $m_i$, $\hat{m}_i$ need to be considered. Hence $0(t^2)$ calls of $PI$ or $0(t^3)$ calls of $N$-ORACLE suffice overall. For computing $T$ grafting may cause a ripple effect. On each occasion the value of $V_j$ may have to be updated for up to $t$ such sets, and hence $t^2$ calls of $RA$ will suffice. Hence $0(t^3)$ calls of $RA$ in the overall algorithm will be enough. □

## 8. REMARKS

In this paper we have considered learning as the process of deducing a program for performing a task, from information that does not provide an explicit description of such a program. We have given precise meaning to this notion of learning and have shown that in some restricted but nontrivial contexts it is computationally feasible.

Consider a world containing robots and elephants. Suppose that one of the robots has discovered a recognition algorithm for elephants that can be meaningfully expressed in $k$-conjunctive normal form. Our Theorem A implies that this robot can communicate its algorithm to the rest of the robot population by simply exclaiming "elephant" whenever one appears.

An important aspect of our approach, if cast in its greatest generality, is that we require the recognition algorithms of the teacher and learner to agree on an overwhelming fraction of only the natural inputs. Their behavior on unnatural inputs is irrelevant, and hence descriptions of all possible worlds are not necessary. If followed to its conclusion, this idea has considerable philosophical implications: A learnable concept is nothing more than a short program that distinguishes some natural inputs from some others. If such a concept is passed on among a population in a distributed manner, substantial variations in meaning may arise. More importantly, what consensus there is will only be meaningful for natural inputs. The behavior of an individual's program for unnatural inputs has no relevance. Hence thought experiments and logical arguments involving unnatural hypothetical situations may be meaningless activities.

The second important aspect of the formulation is that the notion of oracles makes it possible to discuss a whole range of teacher–learner interactions beyond the mere identification of examples. This is significant in the context of artificial intelligence where humans may be willing to go to great lengths to convey their skills to machines but are frustrated by their inability to articulate the algorithms they themselves use in the practice of the skills. We expect that some explicit programming does become essential for transmitting skills that are beyond certain limits of difficulty. The identification of these limits is a major goal of the line of work proposed in this paper.


**REFERENCES**

1. Angluin, D., Smith, C.H. Inductive inference: Theory and methods. *Comput. Surv. 15*, 3 (Sept. 1983), 237–269.
2. Barr, A., and Feigenbaum, E.A. *The Handbook of Artificial Intelligence.* Vol. 2. William Kaufmann, Los Altos, Calif., 1982.
3. Cook, S.A. The complexity of theorem proving procedures. In *Proceedings of 3rd Annual ACM Symposium on Theory of Computing* (Shaker Heights, Ohio, May 3–5). ACM, New York, 1971, 151–158.
4. Duda, R.O., and Hart, P.F. *Pattern Classification and Scene Analysis.* Wiley, New York, 1973.
5. Erdös, P., and Spencer, J. *Probabilistic Methods in Combinatorics.* Academic Press, New York, 1974.
6. Goldreich, O., Goldwasser, S., and Micali, S. How to construct random functions. In *Proceedings of 25th IEEE Symposium on Foundations of Computer Science* (Singer Island, Fla., Oct. 24–26). IEEE, New York, 1984.
7. Michalski, R.S., Carbonell, J.G., and Mitchell, T.M. *Machine Learning: An Artificial Intelligence Approach.* Tioga Publishing Co., Palo Alto, Calif., 1983.
8. Skyum, S., and Valiant, L.G. A complexity theory based on Boolean algebra. In *Proceedings of 22nd IEEE Symposium on Foundations of Computer Science* (Nashville, Tenn., Oct. 28–30). IEEE, New York, 1981, 244–253.
9. Valiant, L.G. Deductive learning. *Philosophical Transactions of the Royal Society of London* (1984). To be published.



**CR Categories and Subject Descriptors:** F.1.1 [**Computation by Abstract Devices**]: Models of Computation; F.2.2 [**Analysis of Algorithms and Problem Complexity**]: Nonnumerical Algorithms and Problems; I.2.6 [**Artificial Intelligence**]: Learning—*concept learning*

**General Terms:** Algorithms

**Additional Key Words and Phrases:** inductive inference, probabilistic models of learning, propositional expressions

Received 1/84; revised 6/84; accepted 7/84



Author's Present Address: L.G. Valiant, Center for Research in Computing Technology, Aiken Computation Laboratory, Harvard University, Cambridge, MA 02138.



Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.






gi, g] são colocados na mesma expressão de te mpos em qualquer fase subsequente, então fi, ~ estão no mesmo produto em f.

Prova: (1) Se as condições de (1) acima forem válidas, e ntão giT& será descoberto na próxima fase temporal e a reivindicação seguirá. (2) Iffi, J~não estão no mesmo produto em f, então f contém algum implicante primo contendo variáveis de um de gi, & e não do outro. Portanto, giTgj nunca será válido.

Prova da Reivindicação 1: Por indução no número de fases nas Reivindicações 1, 3 e 4 simultaneamente.

Definimos a profundidade de uma fórmula fi como o número máximo de alternâncias de soma e produto necessárias na sua definição.

Reivindicação 5: Iffi é uma expressão de profundidade k em f, então após k fases um gi idêntico a fi terá sido construído pelo algoritmo.

Prova: Por indução nas reivindicações 3 e 4.

Para concluir a prova do teorema, resta analisar o tempo de execução. Isto é dominado pelo custo de calcular mi e rhi, que já consideramos, mais o custo de calcular as relações de equivalência S e T. Notamos primeiro que menos de 2t nomes de expressões gi são usados no decorrer do algoritmo. Quando uma expressão é enxertada em outra em um ponto profundo desta última, então a semântica de todas as subexpressões acima mudará. Pelas reivindicações 3 e 4, tal enxerto pode ocorrer quando um gi é adicionado a uma expressão de soma, mas não quando adicionado a uma expressão de tempos. Portanto, os valores mi e rhi não precisam ser alterados para nenhuma expressão quando tal enxerto ocorre. Segue-se que para calcular S apenas 2t valores de m~, rh~precisam ser considerados. Portanto, 0(t 2) chamadas de PI ou 0(t a) chamadas de N-ORACLE são suficientes no geral. Para computação, o enxerto T pode causar um efeito cascata. Em cada ocasião o valor de Vj pode ter que ser atualizado para até t tais conjuntos e, portanto, t2 chamadas de RA serão suficientes. Portanto, 0(t a) chamadas de RA no algoritmo geral serão suficientes. []

8. OBSERVAÇÕES
Neste artigo w

Consideramos a aprendizagem como o processo de deduzir um programa para executar uma tarefa, a partir de informações que não fornecem uma descrição explícita de tal programa. Demos um significado preciso a esta noção de aprendizagem e mostramos que em alguns contextos restritos, mas não triviais, ela é computacionalmente viável.

Considere um mundo contendo robôs e elefantes. Suponha que um dos robôs tenha descoberto um algoritmo de reconhecimento para elefantes que pode ser expresso de forma significativa na forma normal k-conjuntiva. Nosso Teorema A implica que este robô pode comunicar seu algoritmo ao resto da população de robôs simplesmente exclamando " elefante" sempre que um aparece.

Um aspecto importante da nossa abordagem, se apresentada na sua maior generalidade, é que exigimos que os algoritmos de reconhecimento do professor e do aluno concordem sobre uma fracção esmagadora apenas dos inputs naturais. Deles o comportamento com base em informações não naturais é irrelevante e, portanto, descrições de todos os mundos possíveis não são necessárias. Se for seguida até à sua conclusão, esta ideia tem implicações filosóficas consideráveis: um conceito que pode ser aprendido nada mais é do que um programa curto que distingue alguns inputs naturais de o utros. Se tal conceito for transmitido entre uma população de forma distribuída, poderão surgir variações substanciais de significado. Mais importante ainda, qualquer consenso que exista só será significativo para os factores de produção naturais. O comportamento do programa de um indivíduo para insumos não naturais não tem relevância. Portanto, experimentos mentais e argumentos lógicos envolvendo situações hipotéticas não naturais podem ser atividades sem sentido.

O segundo aspecto importante da formulação é que a noção de oráculos torna possível discutir toda uma gama de interações professor-aluno para além da mera identificação de exemplos. Isto é significativo no contexto da inteligência artificial, onde os humanos podem estar dispostos a fazer grandes esforços para transmitir as suas competências às máquinas, mas ficam frustrados pela sua incapacidade de articular os algoritmos que eles próprios utilizam na prática das competências. Esperamos que alguma programação explícita se torne essencial para a transmissão de co mpetências que estão além de certos limites de dificuldade. A identificação desses limites é um objetivo principal da linha de trabalho proposta neste artigo.


REFERÊNCIAS 1. Angluin, D

., Smith, CH. Inferência indutiva: Teoria e métodos. Computação. Sobreviver. 15, 3 (setembro de 1983), 237-269. 2. Barr, A. e Feigenbaum, EA. O Manual de Inteligência Artificial. Vol. 2. William Kaufmann, Los Altos, Califórnia, 1982. 3. Cook, SA A complexidade dos procedimentos de prova de teoremas. In Proceedings of 3rd Annual ACM Symposium on Theory of Computing (Shaker Heights, Ohio, 3 a 5 de maio). ACM, Nova York, 1971, 151-158. 4. Duda, RO e Hart, PF. Classificação de padrões e análise de cena. Wiley, Nova York. 1973. 5. ErdSs, P. e Spencer, J. Métodos Probabilísticos em Combinatória. Academic Press, Nova York, 1974. 6. Goldreich, O., Goldwasser, S., e Micali, S. Como construir funções aleatórias. Em Anais do 25º Simpósio IEEE sobre Fundamentos da Ciência da Computação (Singer Island, Flórida, 24 a 26 de outubro). I EEE, Nova York, 1984. 7. Michalski, RS. Carbonell, JG e Mitchell, T.M. Aprendizado de máquina: uma abordagem de inteligência artificial. Tioga Publishing Co., Palo Alto, Califórnia, 1983. 8. Skyum, S., e Valiant, L.G. Uma teoria da complexidade baseada na álgebra booleana. Em Proceedings of 22rid IEEE Symposium on Foundations of Computer Science (Nashville, Tenn., 28-30 de outubro}. IEEE, New York, 1981, 244-253. 9. Valiant, LG Deduction learning. Philosophical Transactions of the Royal Society of London (1984). A ser publicado.



Categorias CR e Descritores de Assuntos: F.1.1 [Computação por Dispositivos Abstratos]: Modelos de Computação; F.2.2[Análise de Algoritmos e Complexidade de Problemas]: Algoritmos e Problemas Não Numéricos; L2.6[Inteligência Artificial]: Aprendizagem - aprendizagem conceitual Termos Gerais: Algoritmos

Palavras-chave e frases adicionais: inferência indutiva, modelos probabilísticos de aprendizagem, expressões proposicionais

Recebido em 1/84; revisado em 6/84; aceito em 7/84



Endereço atual do autor: L.G. Valiant, Centro de Pesquisa em Tecnologia da Computação. Laboratório de Computação Aiken, Universidade de Harvard, Cambridge, MA 02138.



A permissão para copiar gratuitamente todo ou parte deste material é concedida desde que as cópias não sejam feitas ou distribuídas para vantagem comercial direta, o aviso de direitos autorais da ACM e o título da publicação e sua data apareçam, e seja fornecido aviso de que a cópia é feita com permissão da Association for Computing Machinery. Para copiar de outra forma, ou republicar, é necessária uma taxa e/ou permissão específica.